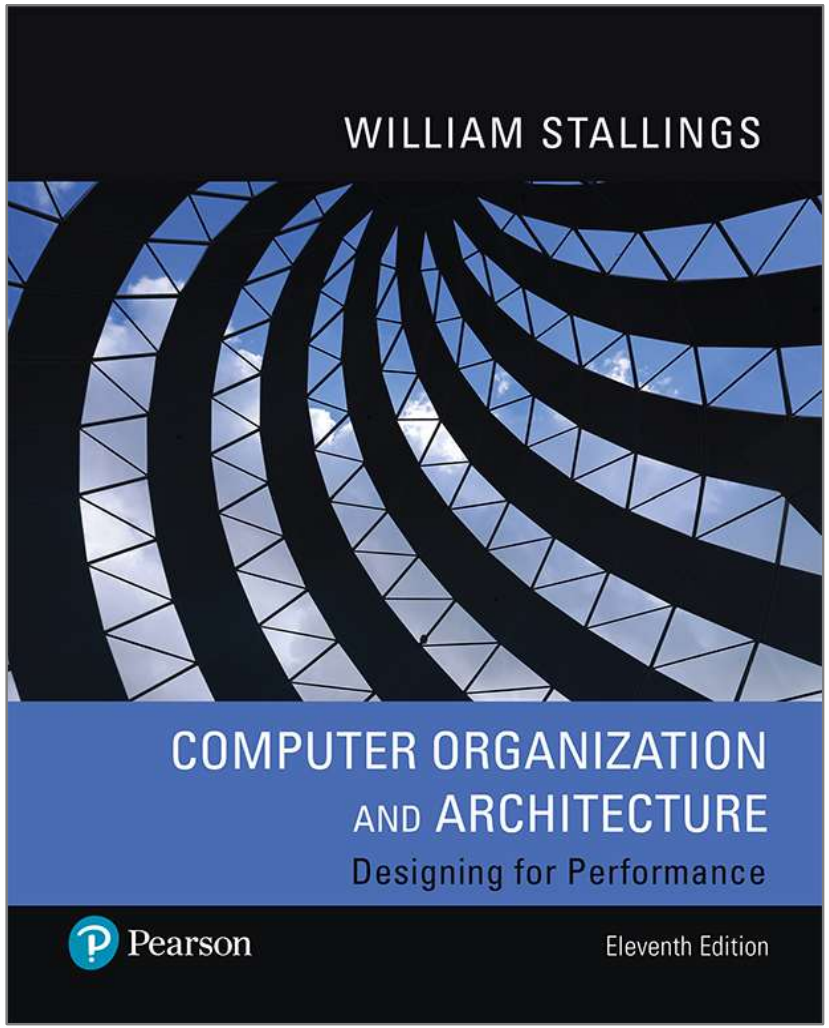
Computer Organization and Architecture

Designing for Performance

11th Edition



Chapter 3

A Top-Level View of Computer Function and Interconnection

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

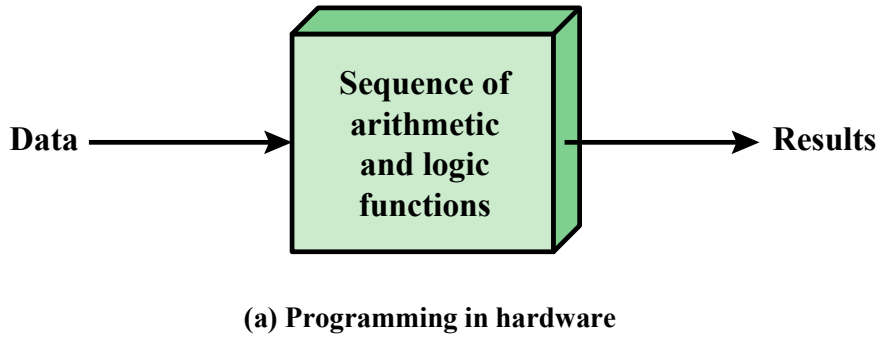
Computer Components

- Contemporary computer designs are based on concepts developed by John von Neumann at the Institute for Advanced Studies, Princeton
- Referred to as the *von Neumann architecture* and is based on three key concepts:
  - Data and instructions are stored in a single read-write memory
  - The contents of this memory are addressable by location, without regard to the type of data contained there
  - Execution occurs in a sequential fashion (unless explicitly modified) from one instruction to the next
- *Hardwired program*
  - The result of the process of connecting the various components in the desired configuration

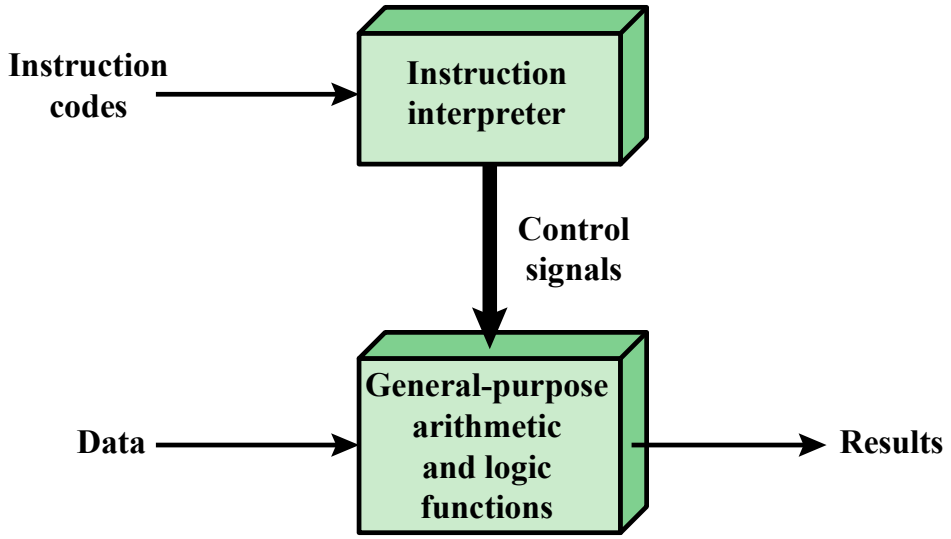
Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Hardware and Software Approaches



(a) Programming in hardware



(b) Programming in software

Figure 3.1 Hardware and Software Approaches

Pearson
Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Software and I/O Components

Software

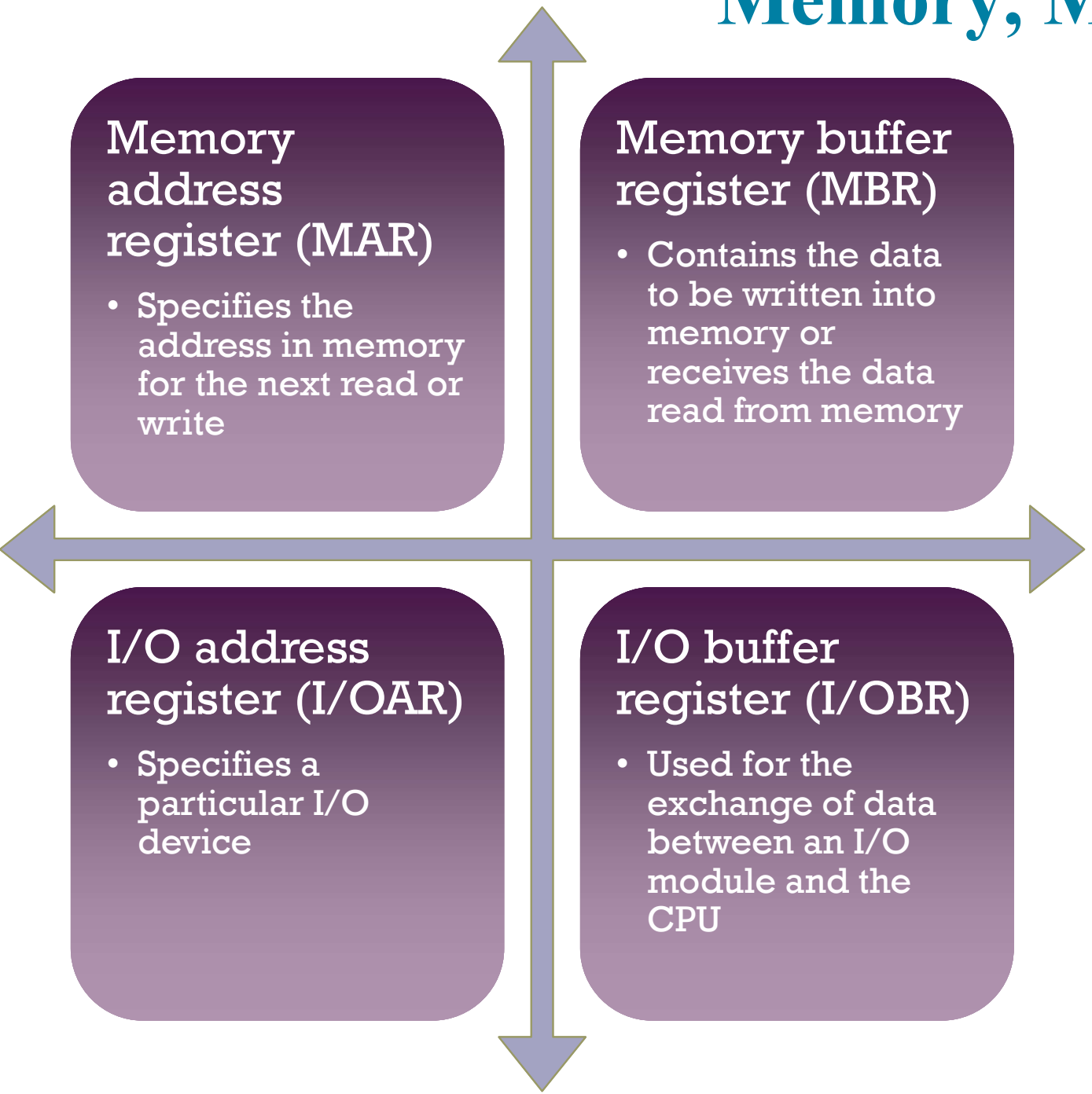
- A sequence of codes or instructions
- Part of the hardware interprets each instruction and generates control signals
- Provide a new sequence of codes for each new program instead of rewiring the hardware

Major components:

- CPU
  - Instruction interpreter
  - Module of general-purpose arithmetic and logic functions
- I/O Components
  - Input module
    - Contains basic components for accepting data and instructions and converting them into an internal form of signals usable by the system
  - Output module
    - Means of reporting results

Pearson
Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Memory, MAR, and MBR



Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Figure 3.2

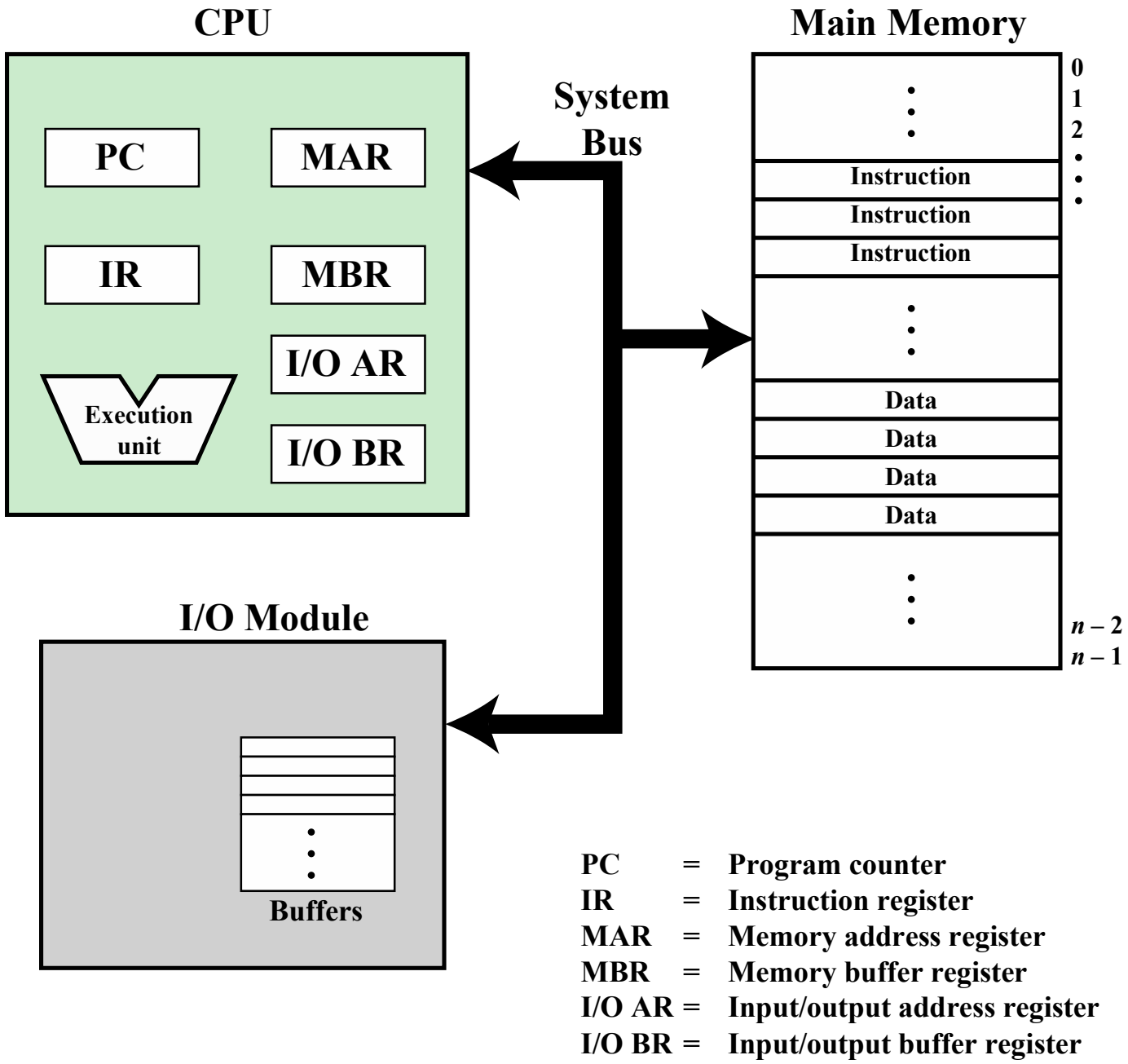


Figure 3.2 Computer Components: Top-Level View

Pearson
Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Figure 3.3

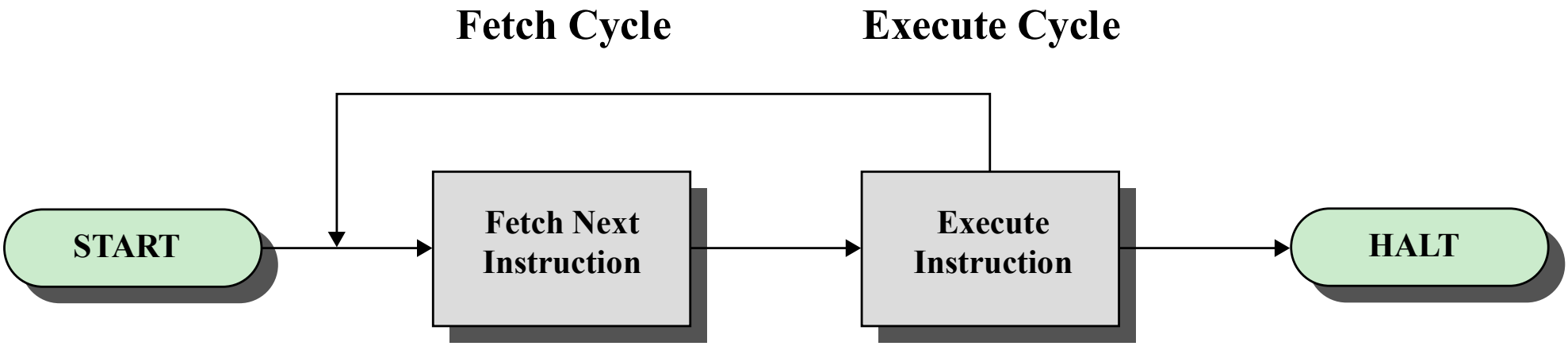


Figure 3.3 Basic Instruction Cycle

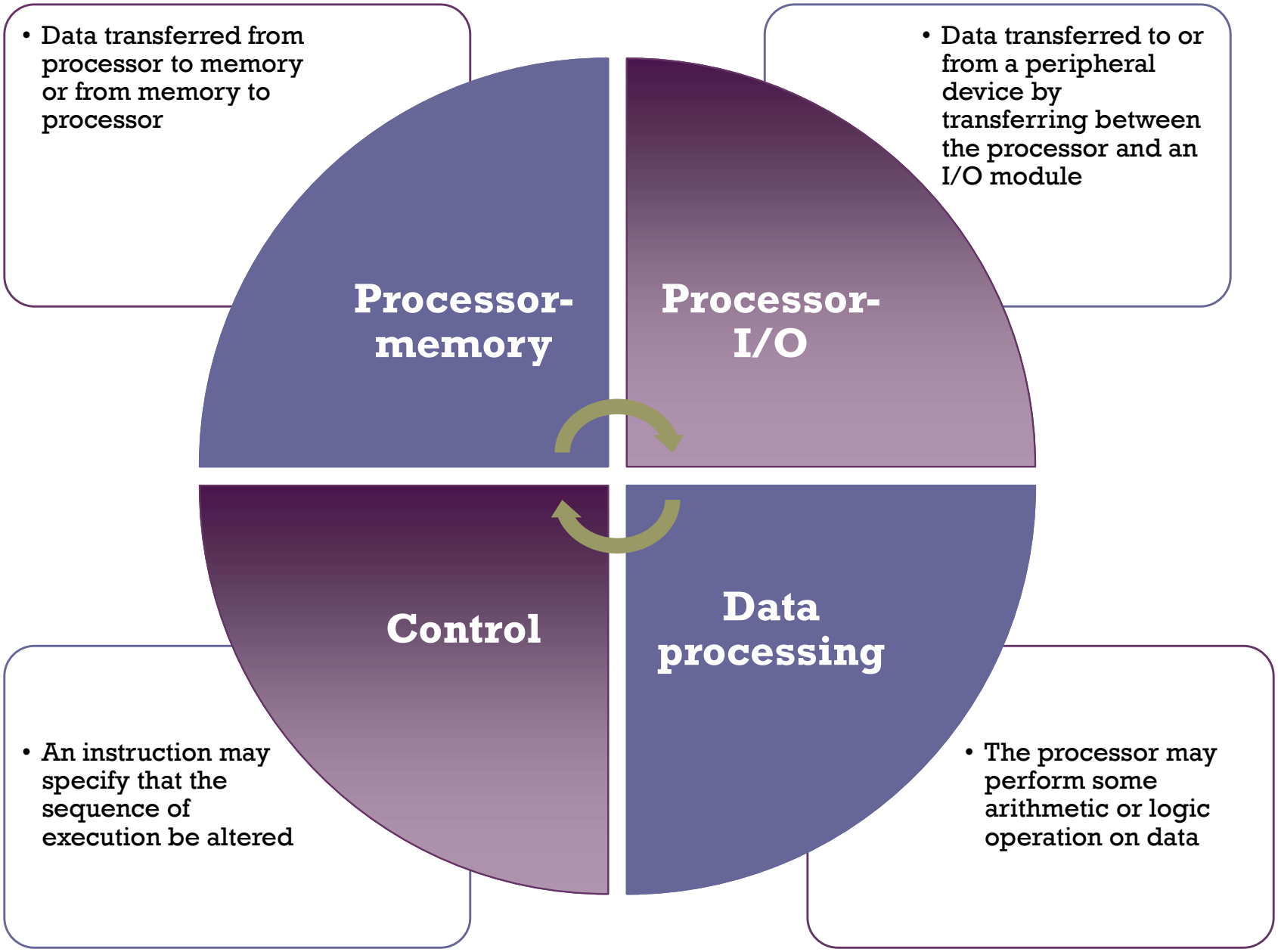
Pearson
Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Fetch Cycle

- At the beginning of each instruction cycle the processor fetches an instruction from memory
- The program counter (PC) holds the address of the instruction to be fetched next
- The processor increments the PC after each instruction fetch so that it will fetch the next instruction in sequence
- The fetched instruction is loaded into the instruction register (IR)
- The processor interprets the instruction and performs the required action

Pearson
Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

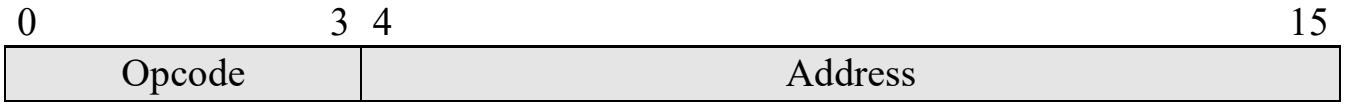
Action Categories



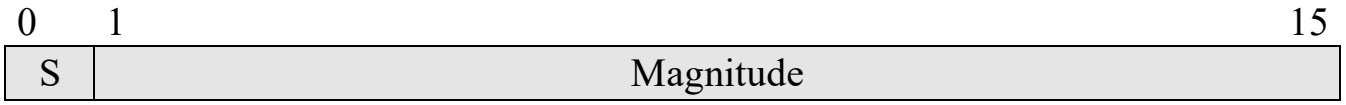
Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Figure 3.4



(a) Instruction format



(b) Integer format

Program Counter (PC) = Address of instruction
Instruction Register (IR) = Instruction being executed
Accumulator (AC) = Temporary storage

(c) Internal CPU registers

0001 = Load AC from Memory
0010 = Store AC to Memory
0101 = Add to AC from Memory

(d) Partial list of opcodes

Figure 3.4 Characteristics of a Hypothetical Machine

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Figure 3.5

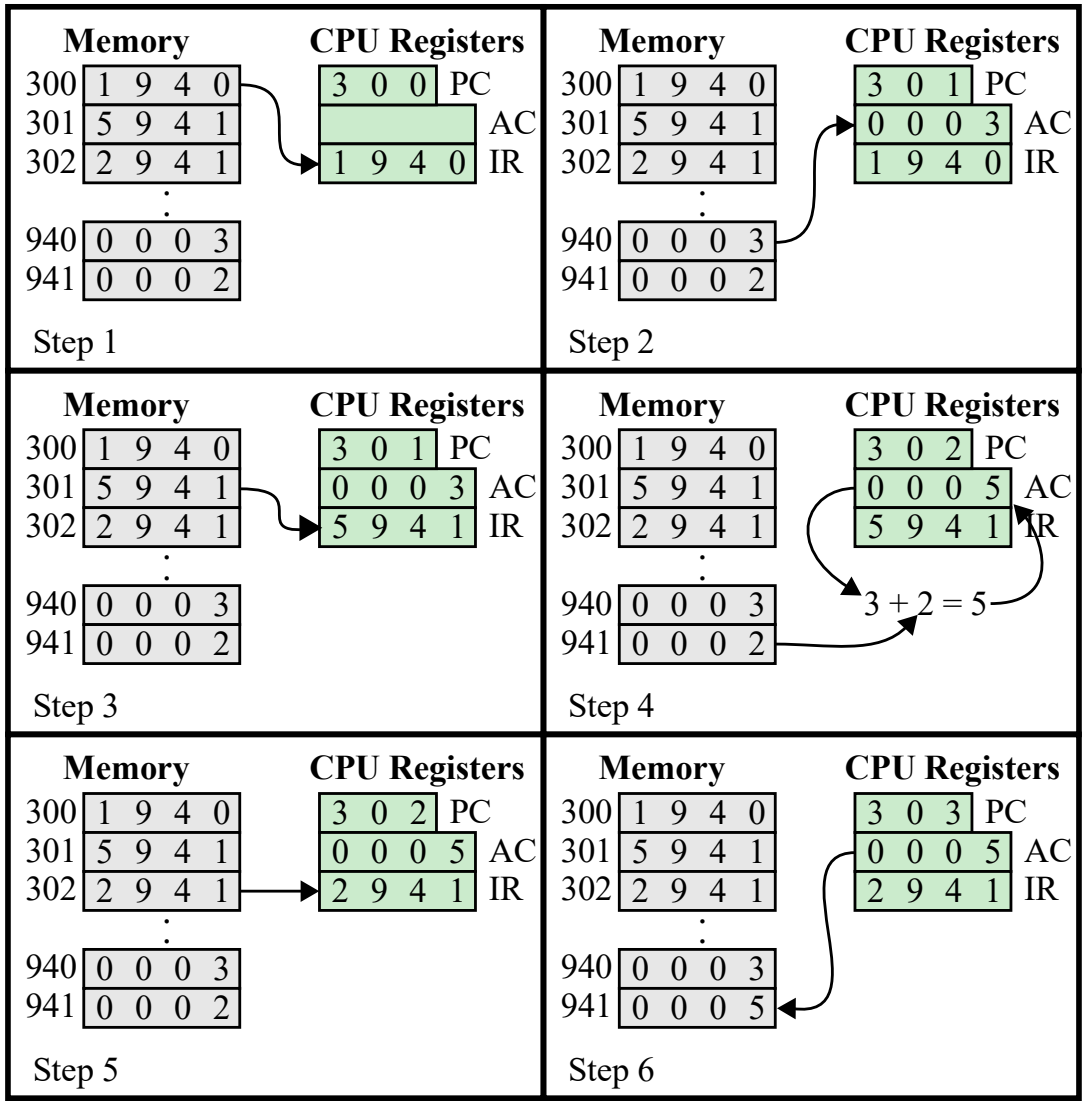


Figure 3.5 Example of Program Execution (contents of memory and registers in hexadecimal)

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Instruction Cycle

- In this example, three instruction cycles, each consisting of a fetch cycle and an execute cycle, are needed to add the contents of location 940 to the contents of 941.
- With a more complex set of instructions, fewer cycles would be needed.
- Some older processors, for example, included instructions that contain more than one memory address.
- Instead of memory references, an instruction may specify an I/O operation.

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Instruction Cycle

- Consider PDP-11 processor include an instruction:
  - ADD B, A
- A single instruction cycle with the following steps will occur:
  - Fetch the ADD instruction
  - Read memory location A into the processor
  - Read memory location B into the processor. For content A not loss, processor need 2 register to store values.
  - Add 2 values
  - Write result to location A
- Execution cycle involve more than 1 memory reference. Figure 3.6 illustrate the basic instruction cycle more detail

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Figure 3.6

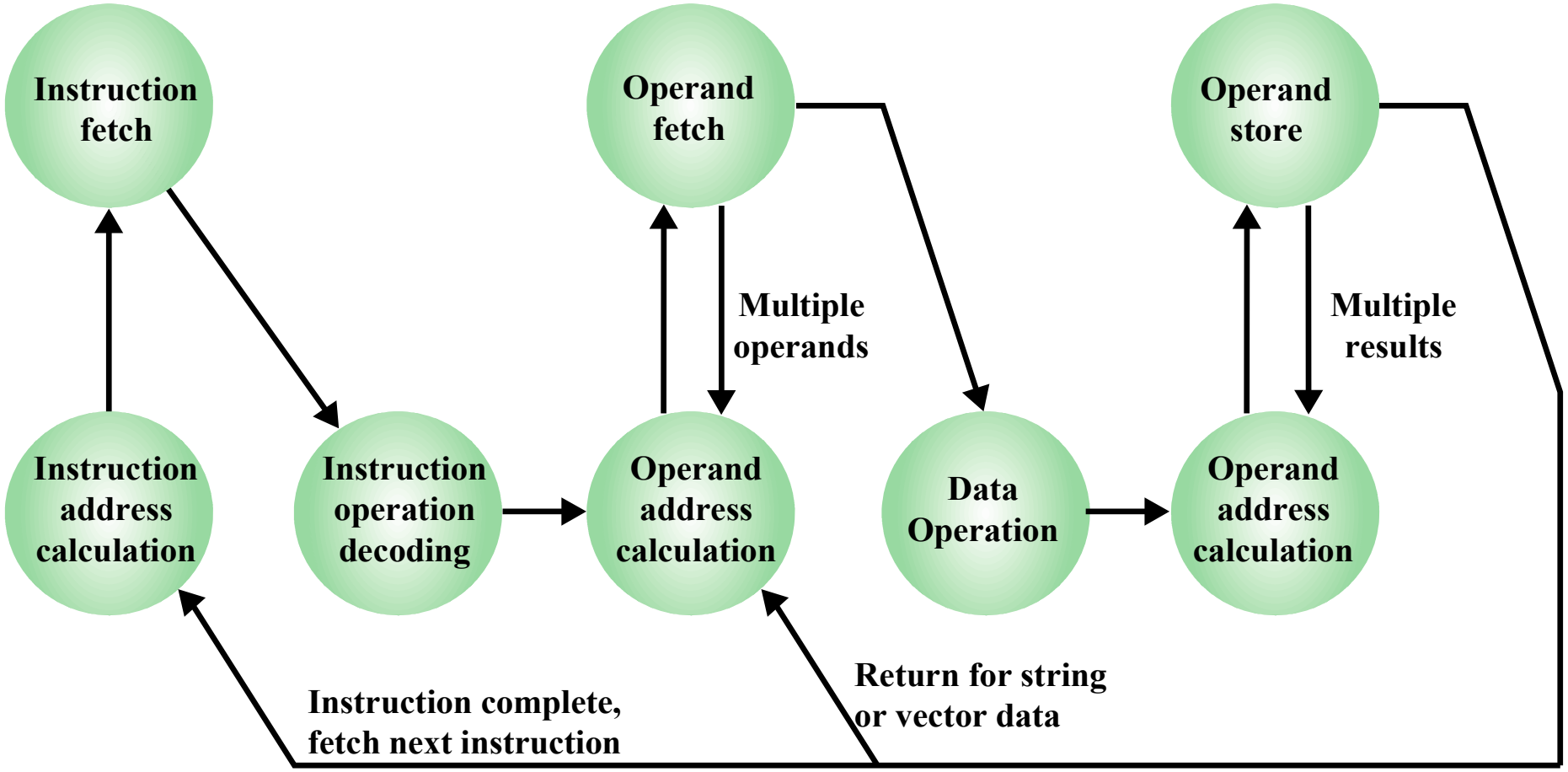


Figure 3.6 Instruction Cycle State Diagram

Pearson
Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Interrupt

- Virtually all computers provide a mechanism by which other modules (I/O, memory) may **interrupt** the normal processing of the processor
- Most common classes of interrupts: Program, Timer, I/O and Hardware Failure

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Table 3.1 Classes of Interrupts

Program	Generated by some condition that occurs as a result of an instruction execution, such as arithmetic overflow, division by zero, attempt to execute an illegal machine instruction, or reference outside a user's allowed memory space.
Timer	Generated by a timer within the processor. This allows the operating system to perform certain functions on a regular basis.
I/O	Generated by an I/O controller, to signal normal completion of an operation, request service from the processor, or to signal a variety of error conditions.
Hardware Failure	Generated by a failure such as power failure or memory parity error.

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Interrupt Process

- Interrupts are provided primarily as a way to improve processing efficiency.
- Most external devices are much slower than the processor.
- Processor is transferring data to a printer using the instruction cycle: After write operation, the processor must pause and remain idle.
- Length of this pause may be in thousands of instruction cycles that do not involve memory. Wasteful use of the processor.
- Figure 3.7a illustrates this scenario

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Figure 3.7

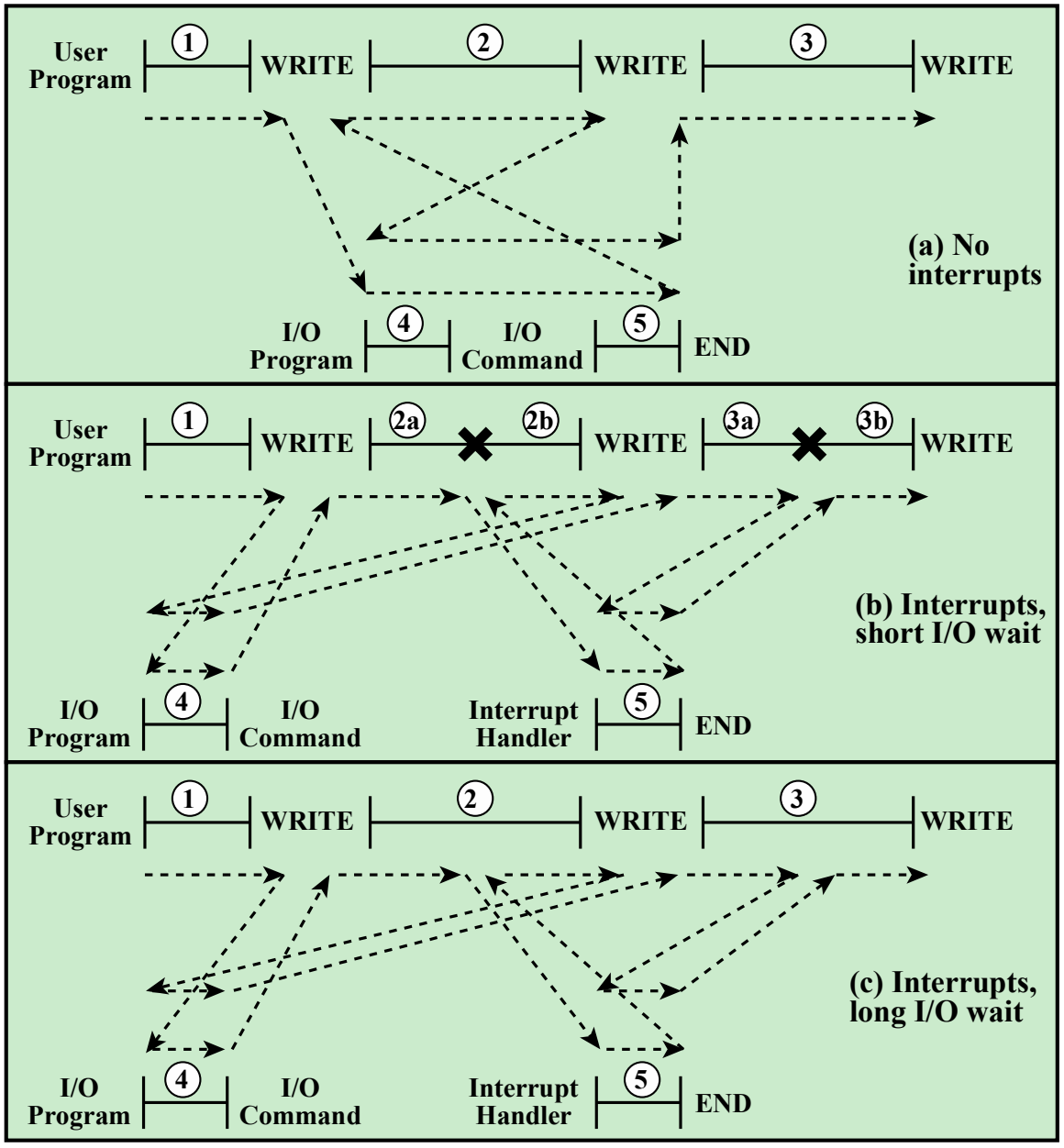


Figure 3.7 Program Flow of Control Without and With Interrupts

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

No Interrupt Scenario

- The user program performs a series of WRITE calls interleaved with processing.
- Code segments 1, 2, and 3 refer to sequences of instructions that do not involve I/O.
- The WRITE calls are to an I/O: perform the actual I/O operation:
  - A sequence of instructions (4): Prepare the actual I/O operation- Copy data to buffer for output etc.
  - The actual I/O command: Without the use of interrupts, once this command is issued, the program must wait for the I/O device to perform the requested function
  - A sequence of instructions (5): complete the operation by setting up the flag bit.

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

With Interrupt Scenario

- The processor can be engaged in executing other instructions while an I/O operation is in progress. Figure 3.7b illustrate program with interrupt.
- Previously, the user program reaches a point for WRITE call. The I/O program that is invoked - preparation code and the actual I/O command.
- External device is busy accepting data from computer memory and printing it. This I/O operation is conducted concurrently with the execution of instructions in the user program.

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

With Interrupt Scenario (2)

- When the external device becomes ready - device sends an *interrupt request* signal to the processor. The processor responds by suspending operation of the current program, branching off to a program to service.
- That particular I/O device, known as an **interrupt handler,** and resuming the original execution after the device is serviced.
- An interrupt is just that: an interruption of the normal sequence of execution.
- The interrupt processing is completed, execution resumes (Figure 3.8).

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Figure 3.8

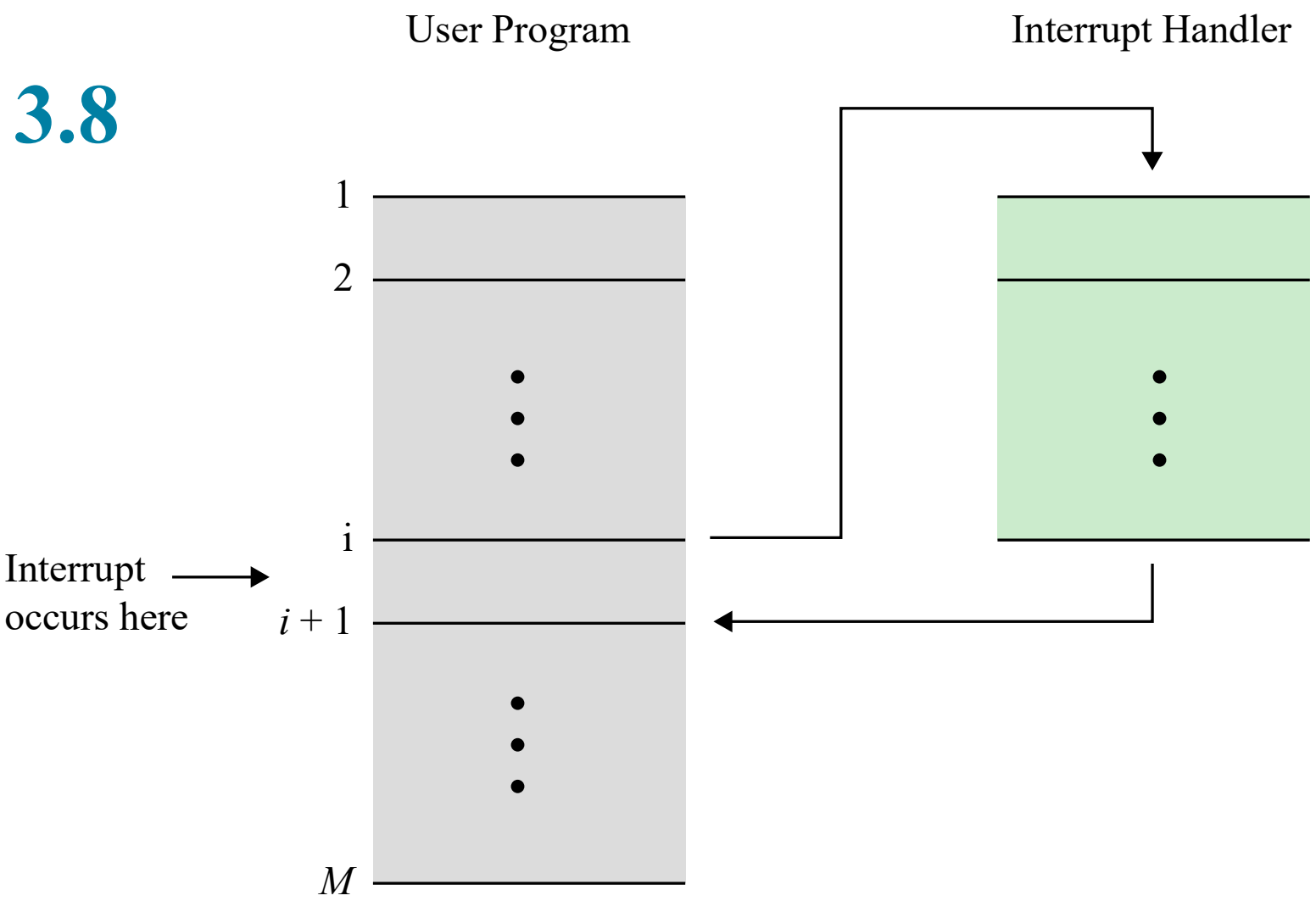


Figure 3.8 Transfer of Control via Interrupts

Pearson
Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Figure 3.9

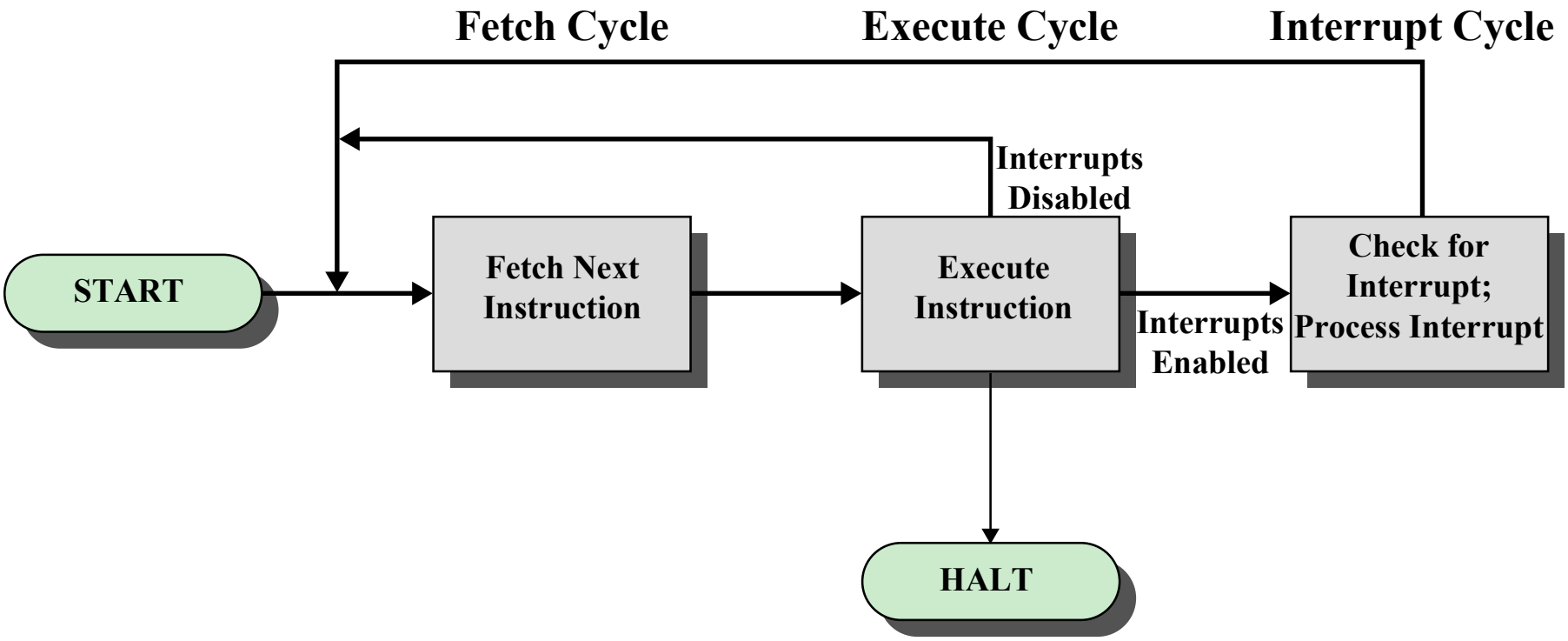


Figure 3.9 Instruction Cycle with Interrupts

Pearson
Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Instruction Cycle with Interrupt

- If interrupts have occurred, the processor does the following:
  - suspends execution of the current program being executed and saves its context.
  - sets the program counter to the starting address of an *interrupt handler routine.*
  - proceeds to the fetch cycle and fetches the first instruction in the interrupt handler program
- *interrupt handler routine*
  - generally part of the operating system. Typically, this program determines the nature of the interrupt and performs whatever actions are needed.
  - In the example we have been using, the handler determines which I/O module generated the interrupt - will write more data out to that I/O module.
  - When the interrupt handler routine is completed, the processor can resume execution of the user program at the point of interruption.

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Instruction Cycle with Interrupt

- T here is some overhead involved in this process. Extra instructions must be executed (in the interrupt handler) to determine the nature of the interrupt and to decide on the appropriate action.
- I/O Operation : Short I/O wait and Long I/O wait
  - The processor must wait while an I/O operation is performed. Code segment no 2 is divide in two during I/O operation
  - Long I/O wait (printer device) (Figure 3.7c) In this case, the user program reaches the second WRITE call before the I/O operation spawned by the first call is complete. The result is that the user program is hung up at that point. When the preceding I/O operation is completed, this new WRITE call may be processed, and a new I/O operation may be started. See Figure 3.11

Pearson
Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Figure 3.10

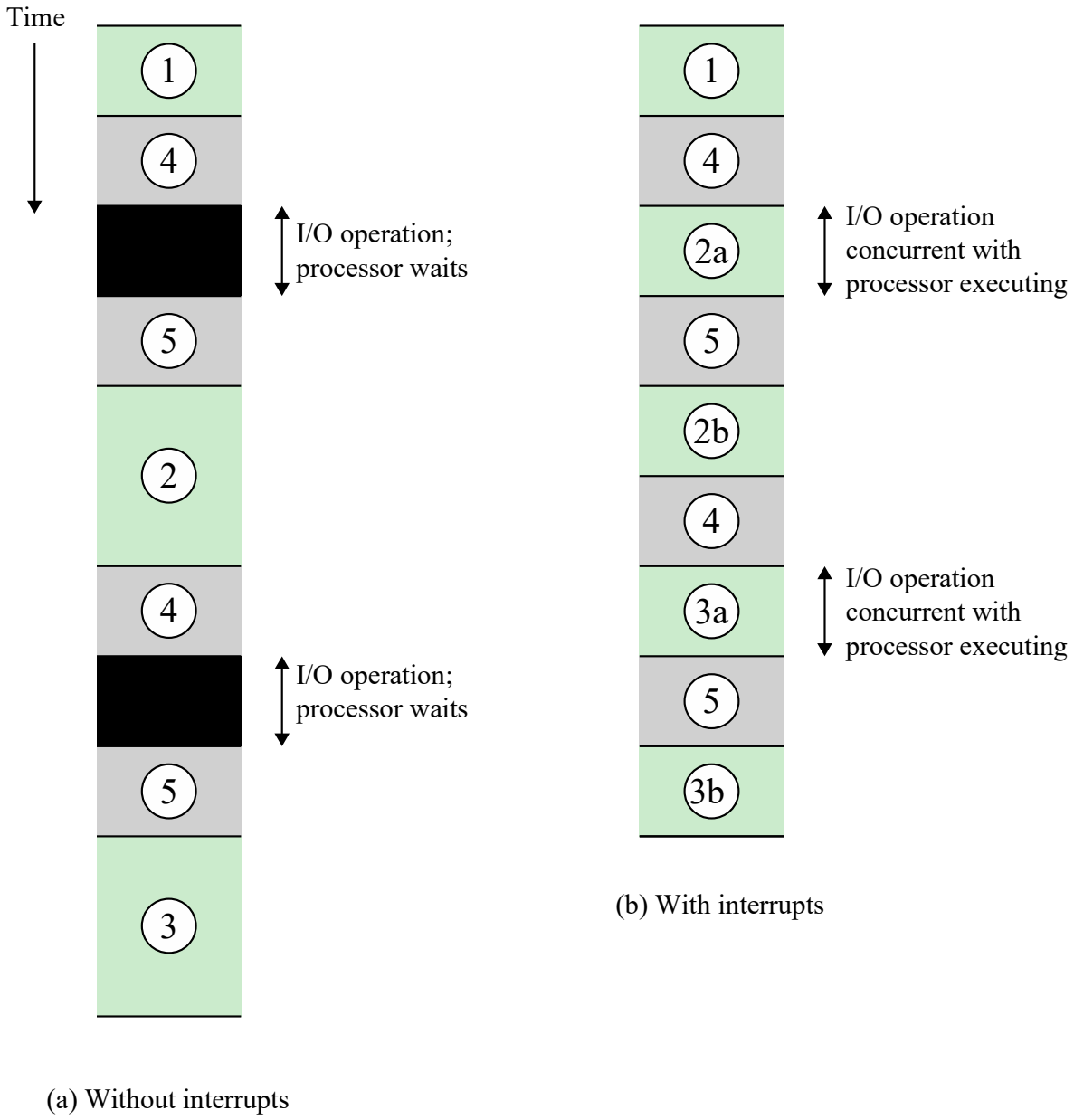


Figure 3.10 Program Timing: Short I/O Wait

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Figure 3.11

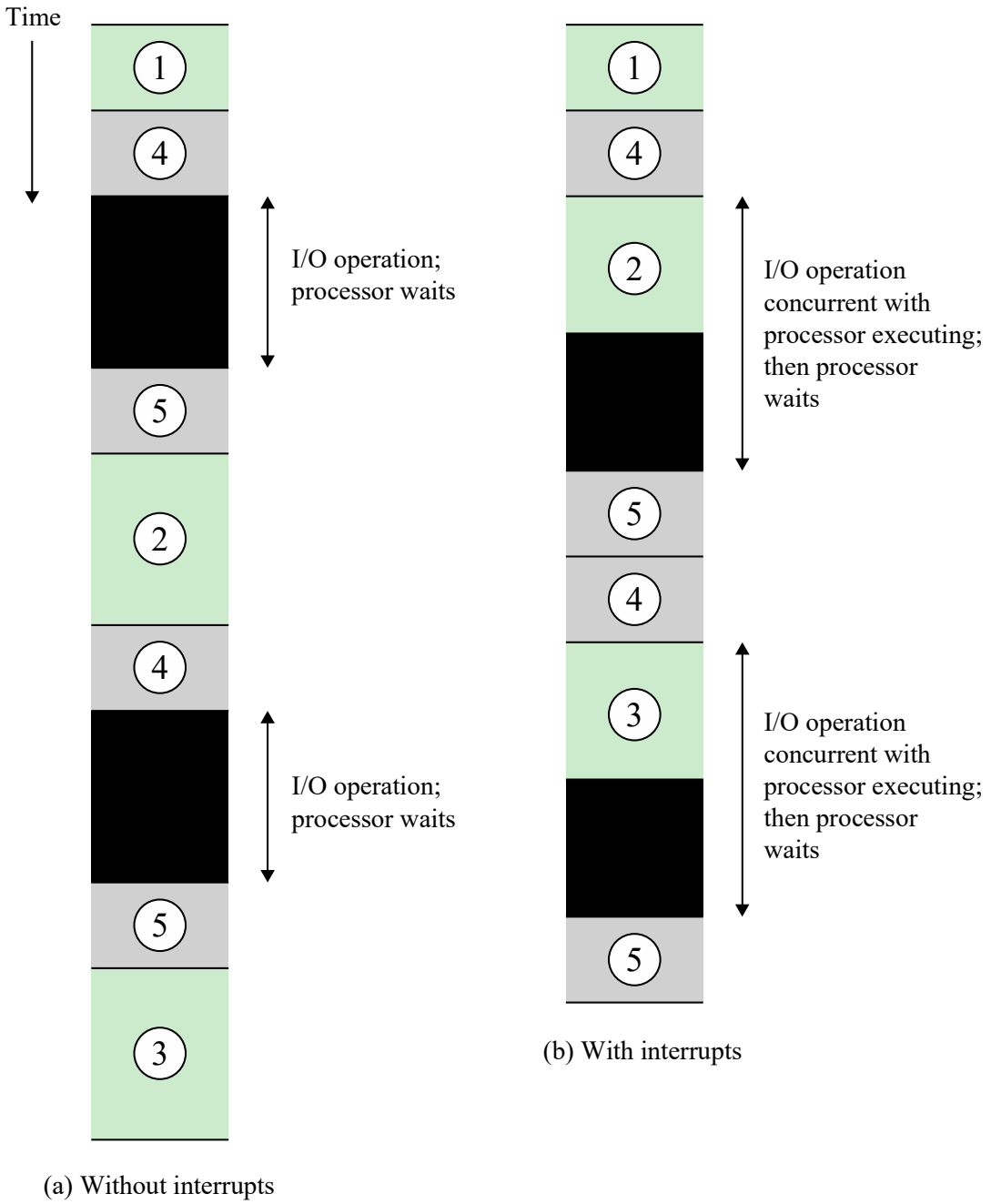


Figure 3.11 Program Timing: Long I/O Wait

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Figure 3.12

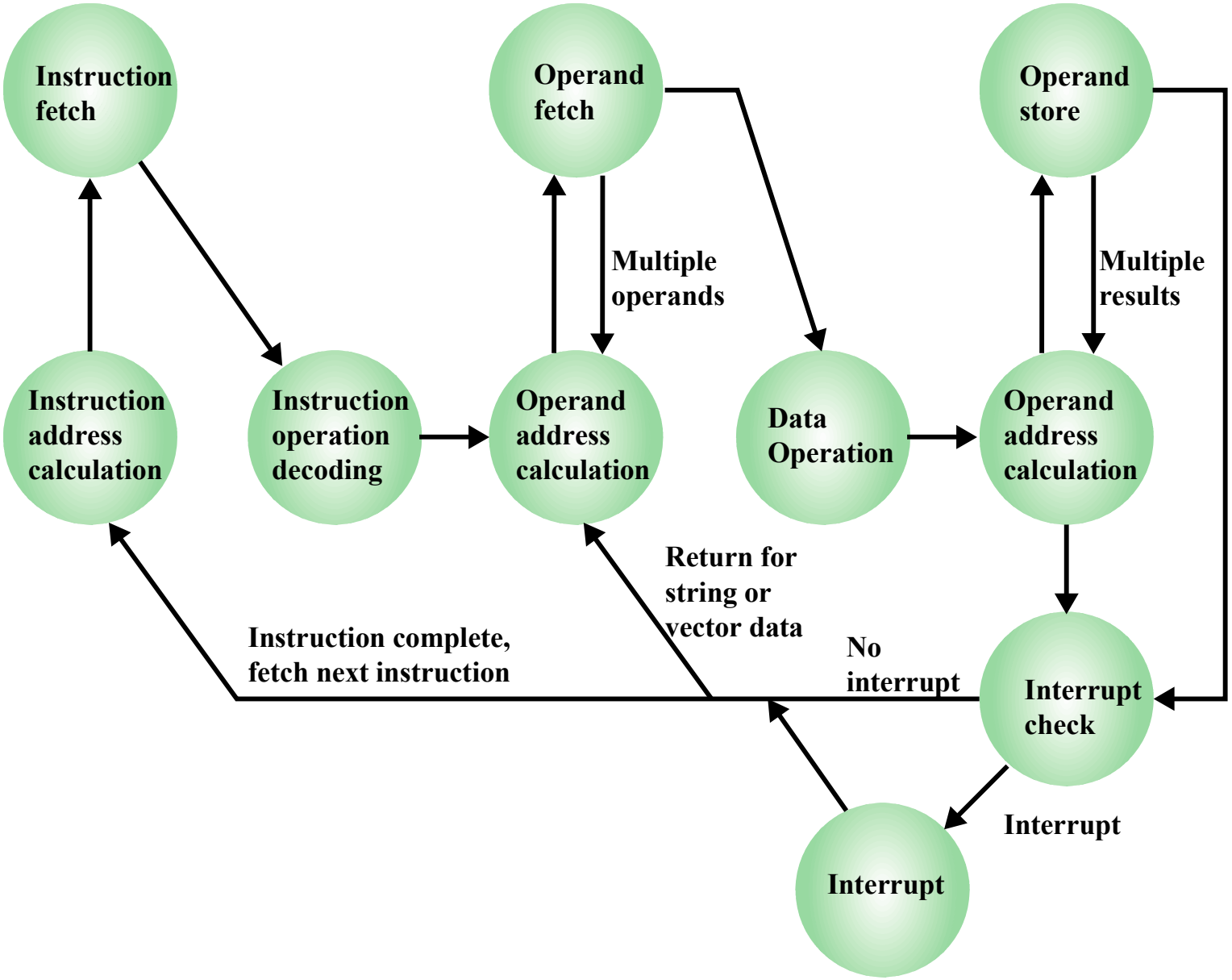


Figure 3.12 Instruction Cycle State Diagram, With Interrupts

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

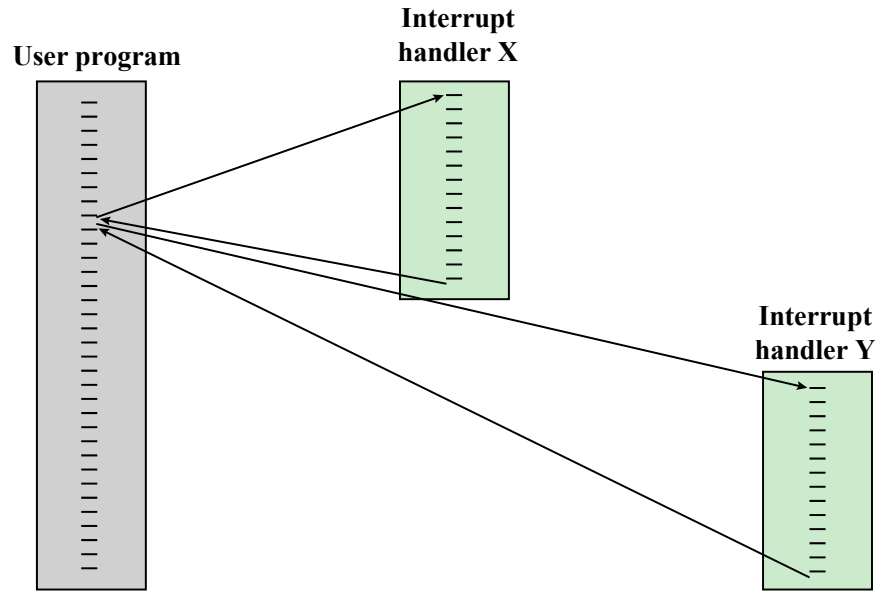
Multiple Interrupt

- Two approaches can be taken to dealing with multiple interrupts.
  - **disabled interrupt** simply means that the processor ignore that interrupt request signal. Generally remains pending and will be checked by the processor after the processor has enabled interrupts. When an interrupt occurs, interrupts are disabled immediately. After the interrupt handler routine completes, interrupts are enabled before resuming the user program, and the processor checks to see if additional interrupts have occurred.
  - The drawback to the preceding approach is that it does not take into account relative priority or time-critical needs.
  - Second approach: to define priorities for interrupts and to allow an interrupt of higher priority

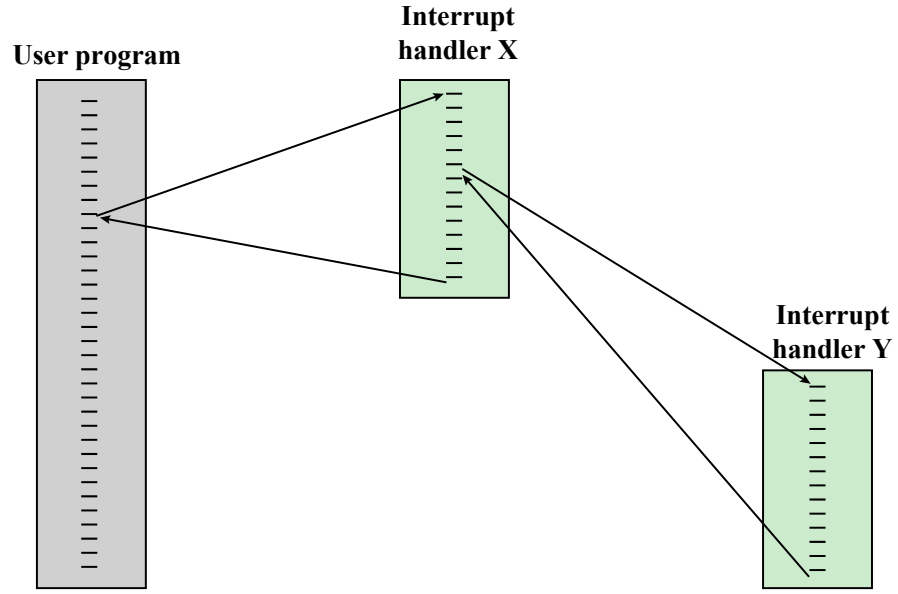
Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Figure 3.13



(a) Sequential interrupt processing



(b) Nested interrupt processing

Figure 3.13 Transfer of Control with Multiple Interrupts

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Figure 3.14

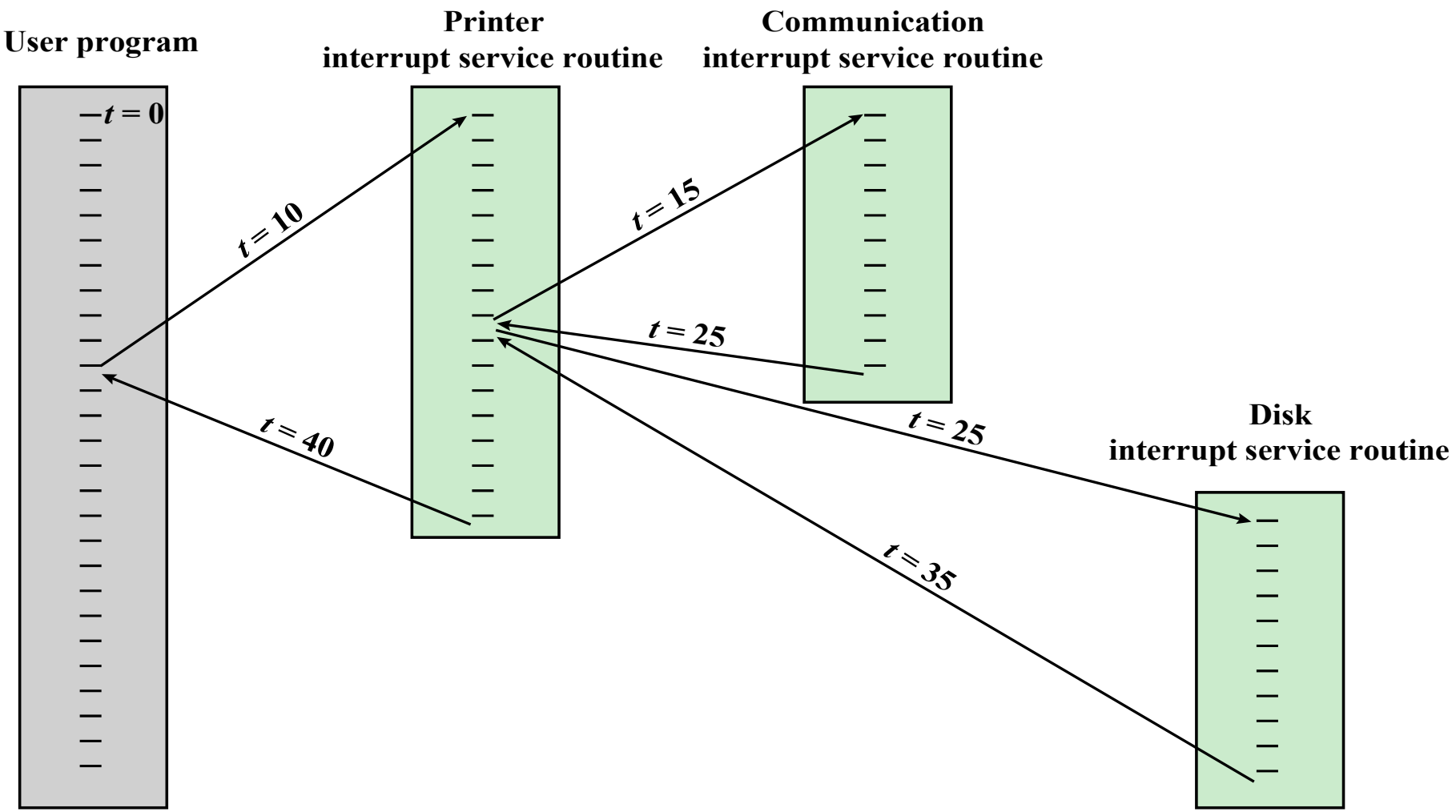


Figure 3.14 Example Time Sequence of Multiple Interrupts

Pearson
Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Interrupt Service Routine (ISR)

- Consider a system with three I/O devices: a printer, a disk, and a communications line, with increasing priorities of 2, 4, and 5, respectively.
- A user program begins at $t = 0$. *At* $t = 10$, a printer interrupt occurs; user information is placed on the system stack and execution continues at the printer **interrupt service routine (ISR).**
- While this routine is still executing, at $t = 15$, *a* communications interrupt occurs. Because the communications line has higher priority than the printer, the interrupt is honored.

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Interrupt Service Routine (ISR)

- The printer ISR is interrupted, its state is pushed onto the stack, and execution continues at the communications ISR. While this routine is executing, a disk interrupt occurs (*t = 20). I*nterrupt is of lower priority, ISR runs to completion.
- When the communications ISR is complete (*t = 25), the previous processor* state is restored, which is the execution of the printer ISR. However, before even a single instruction in that routine can be executed, the processor honors the higher priority disk interrupt and control transfers to the disk ISR.
- When that routine is complete (*t = 35)* the printer ISR resumed. When routine completes (t = 40), control finally returns to the user program.

Pearson
Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

I/O Function

- I/O module can exchange data directly with the processor
- Processor can read data from or write data to an I/O module
  - Processor identifies a specific device that is controlled by a particular I/O module
  - I/O instructions rather than memory referencing instructions
- In some cases it is desirable to allow I/O exchanges to occur directly with memory
  - The processor grants to an I/O module the authority to read from or write to memory so that the I/O memory transfer can occur without tying up the processor
  - The I/O module issues read or write commands to memory relieving the processor of responsibility for the exchange
  - This operation is known as direct memory access (DMA)

Pearson
Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Figure 3.15

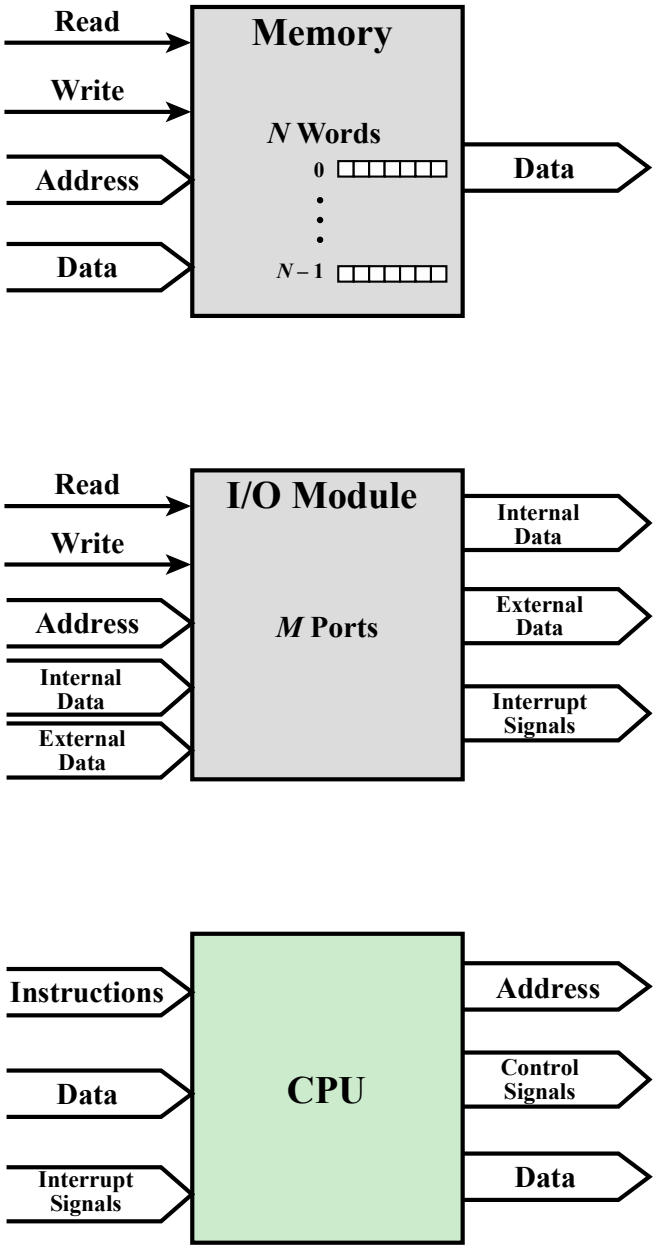


Figure 3.15 Computer Modules

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

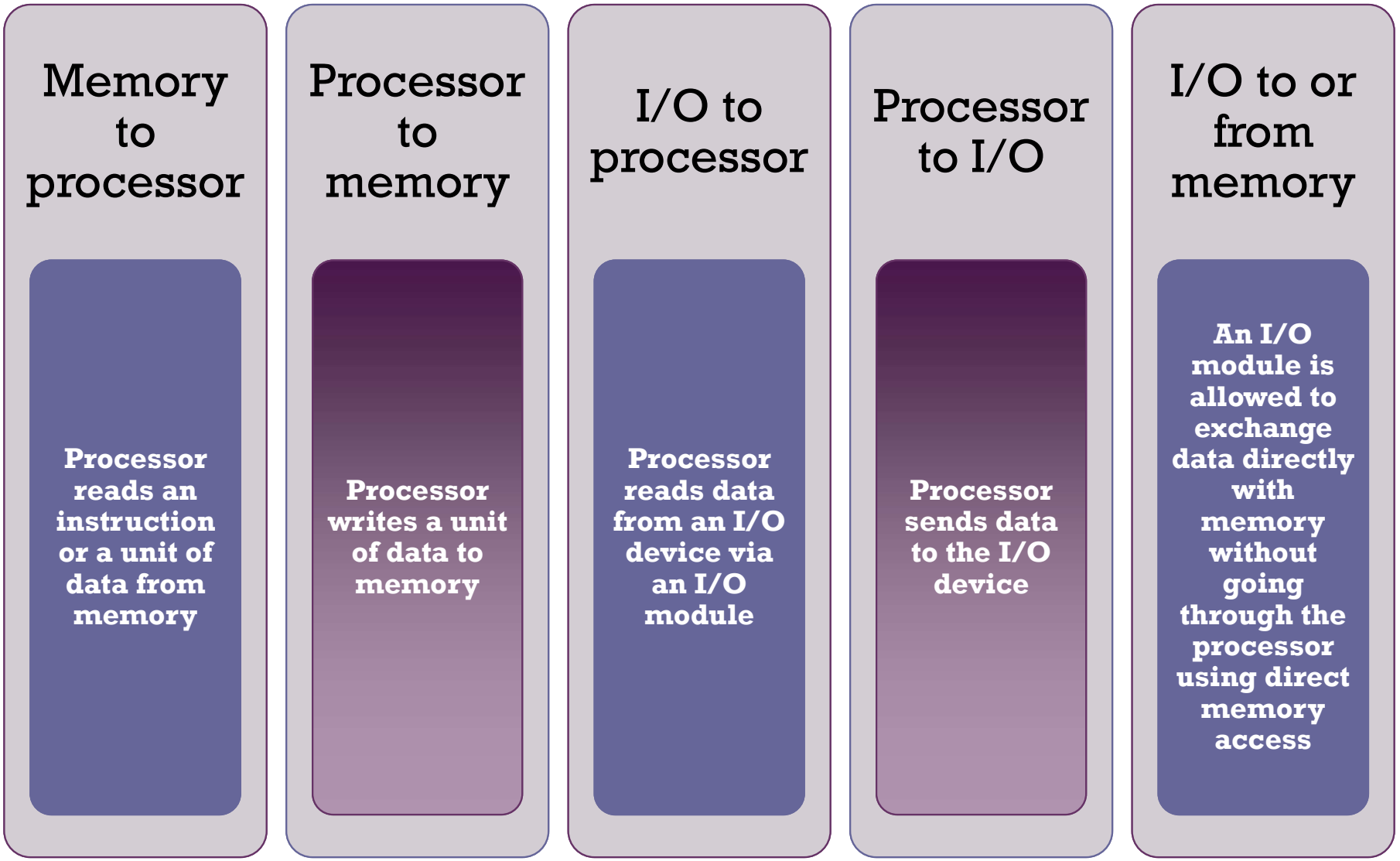
Computer Components

- A computer consists of a set of components or modules of three basic types (processor, memory, I/O) that communicate with each other.
  - **Memory:** Typically, a memory module will consist of *N words of equal length.* Each word is assigned a unique numerical address (0, 1, …, *N - 1).* A word of data can be read from or written into the memory. The location for the operation is specified by an address.
  - **I/O module:** From an internal (to the computer system) point of view, I/O is functionally similar to memory. There are two operations, read and write. Further, an I/O module may control more than one external device.
  - **Processor:** The processor reads in instructions and data, writes out data after processing, and uses control signals to control the overall operation of the system. It also receives interrupt signals.

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

The interconnection structure must support the following types of transfers:



Pearson
Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Bus Interconnection

A communication pathway connecting two or more devices

- Key characteristic is that it is a shared transmission medium

Signals transmitted by any one device are available for reception by all other devices attached to the bus

- If two devices transmit during the same time period their signals will overlap and become garbled

Typically consists of multiple communication lines

- Each line is capable of transmitting signals representing binary 1 and binary 0

Computer systems contain a number of different buses that provide pathways between components at various levels of the computer system hierarchy

System bus

- A bus that connects major computer components (processor, memory, I/O)

The most common computer interconnection structures are based on the use of one or more system buses

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Data Bus

- Data lines that provide a path for moving data among system modules
- May consist of 32, 64, 128, or more separate lines
- The number of lines is referred to as the *width* of the data bus
- The number of lines determines how many bits can be transferred at a time
- The width of the data bus is a key factor in determining overall system performance

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Address Bus

- Used to designate the source or destination of the data on the data bus
  - If the processor wishes to read a word of data from memory it puts the address of the desired word on the address lines
- Width determines the maximum possible memory capacity of the system
- Also used to address I/O ports
  - The higher order bits are used to select a particular module on the bus and the lower order bits select a memory location or I/O port within the module

Control Bus

- Used to control the access and the use of the data and address lines
- Because the data and address lines are shared by all components there must be a means of controlling their use
- Control signals transmit both command and timing information among system modules
- Timing signals indicate the validity of data and address information
- Command signals specify operations to be performed

Pearson
Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Figure 3.16

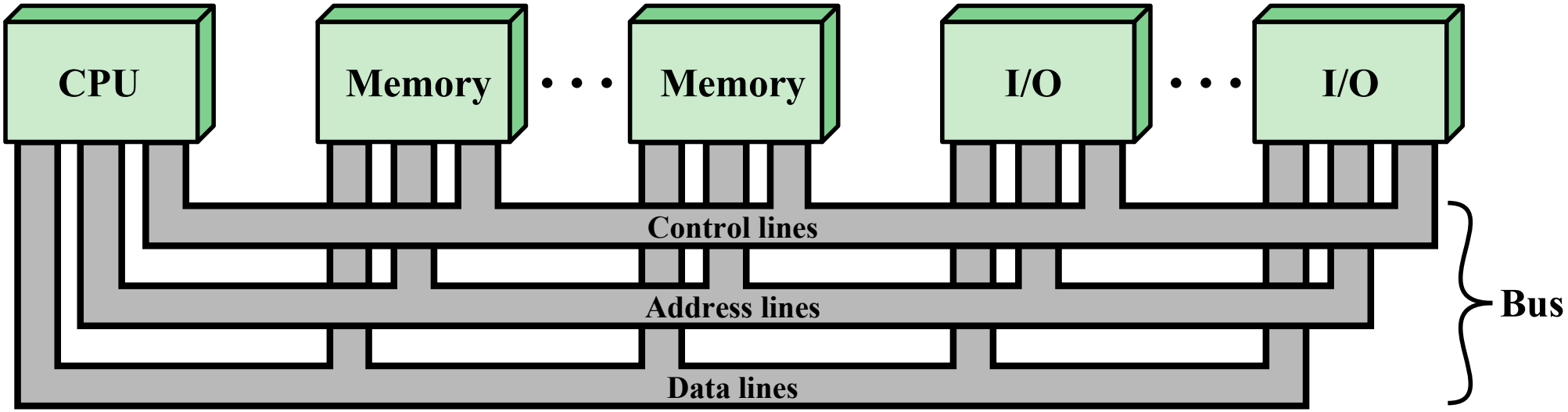


Figure 3.16 Bus Interconnection Scheme

Pearson
Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Point-to-Point Interconnect

Principal reason for change was the electrical constraints encountered with increasing the frequency of wide synchronous buses

At higher and higher data rates it becomes increasingly difficult to perform the synchronization and arbitration functions in a timely fashion

A conventional shared bus on the same chip magnified the difficulties of increasing bus data rate and reducing bus latency to keep up with the processors

Has lower latency, higher data rate, and better scalability

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Quick Path Interconnect

QPI

- Introduced in 2008
- Multiple direct connections
  - Direct pairwise connections to other components eliminating the need for arbitration found in shared transmission systems
- Layered protocol architecture
  - These processor level interconnects use a layered protocol architecture rather than the simple use of control signals found in shared bus arrangements
- Packetized data transfer
  - Data are sent as a sequence of packets each of which includes control headers and error control codes

Pearson
Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Figure 3.17

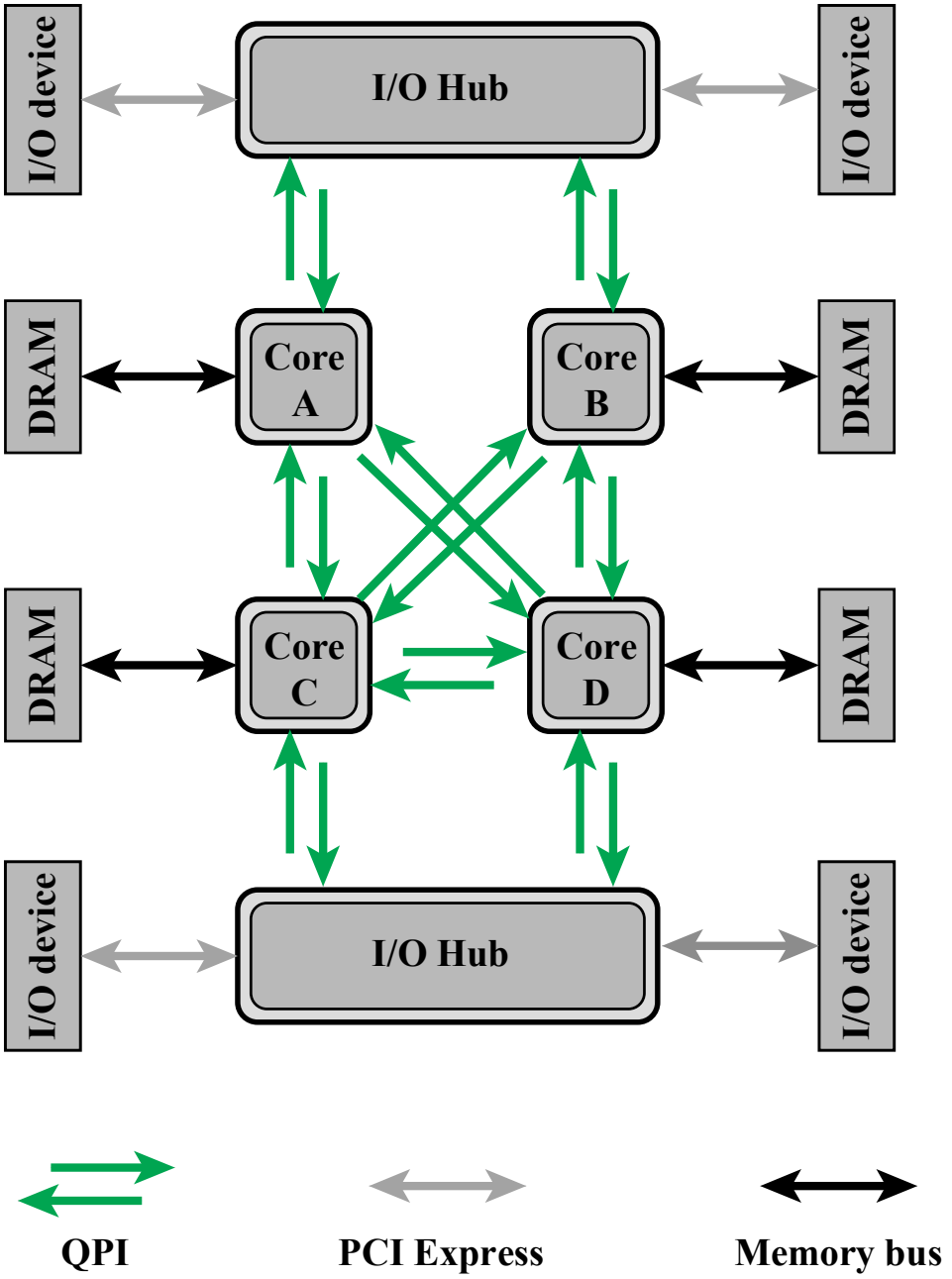


Figure 3.17 Multicore Configuration Using QPI

Pearson
Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Figure 3.18

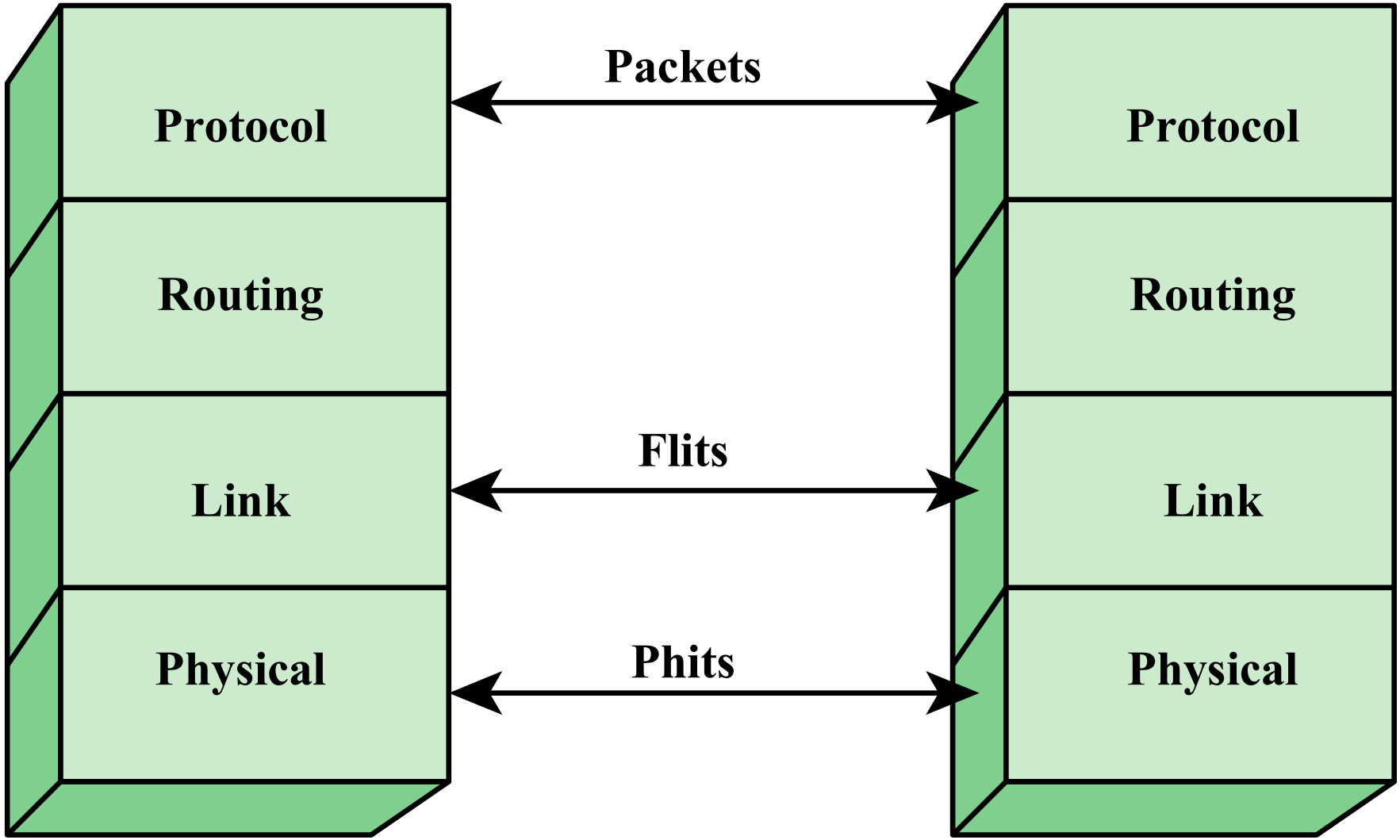


Figure 3.18 QPI Layers

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Figure 3.19

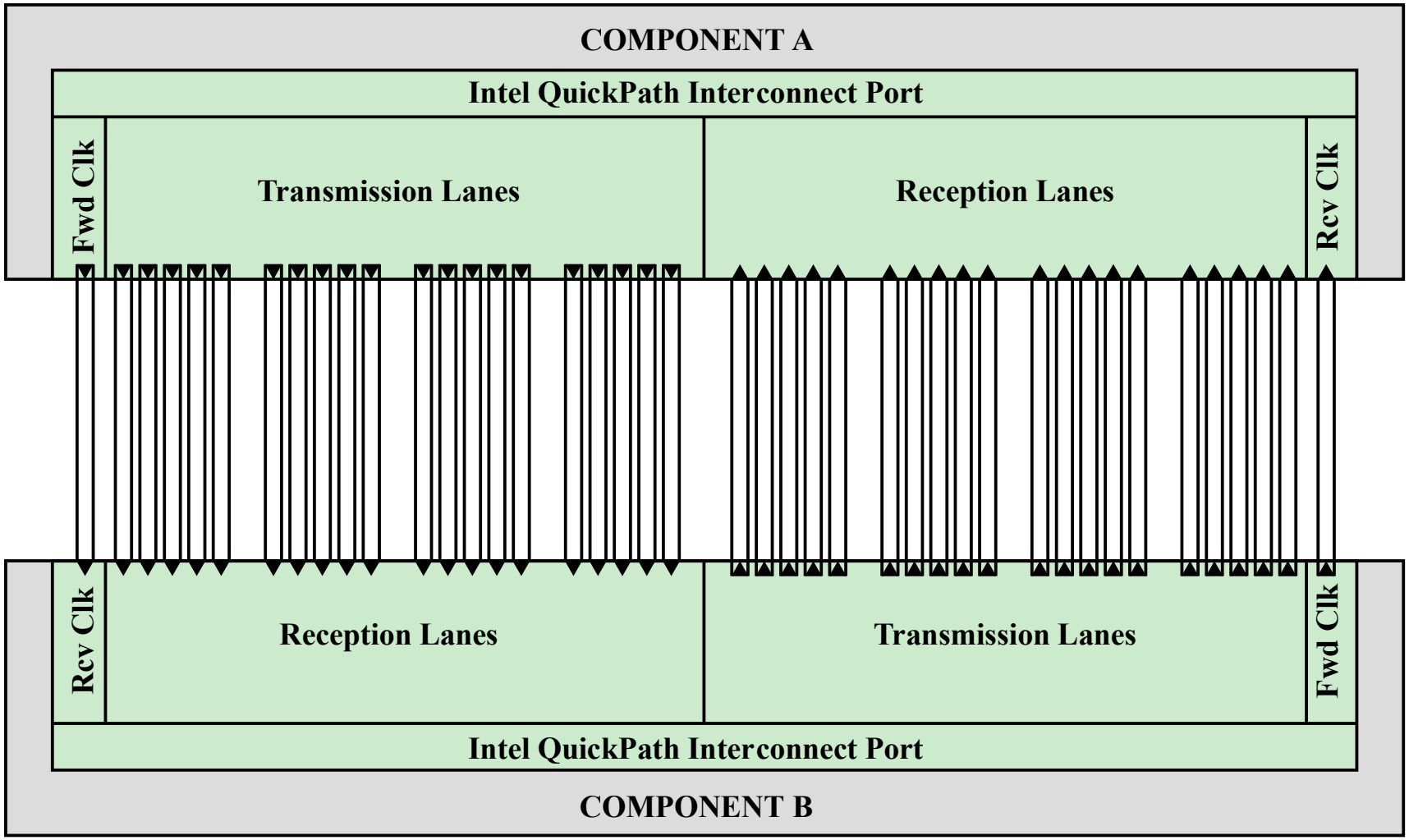


Figure 3.19 Physical Interface of the Intel QPI Interconnect

Pearson
Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Figure 3.20

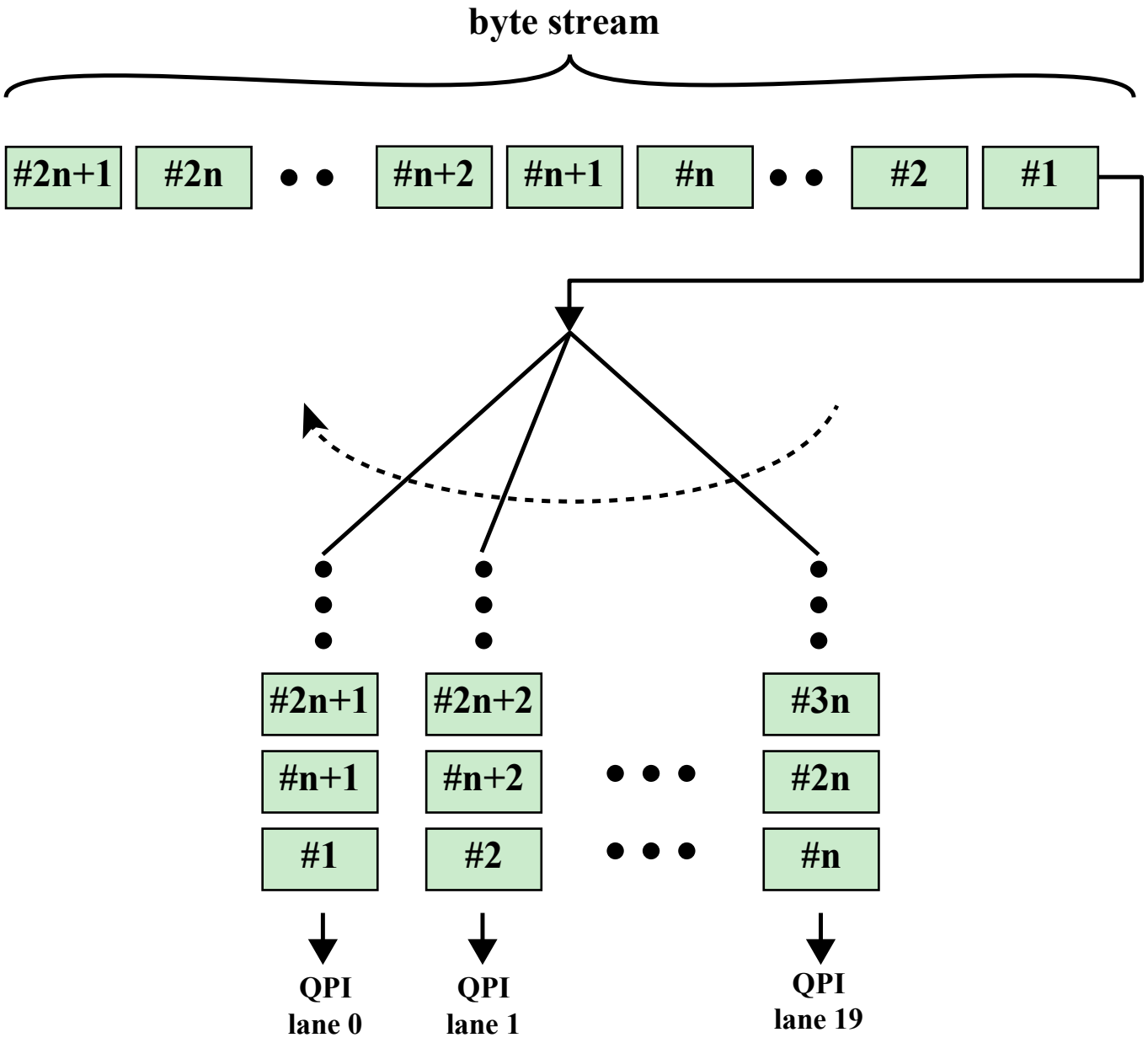


Figure 3.20 QPI Multilane Distribution

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

QPI Link Layer

- Performs two key functions: *flow control* and *error control*
  - Operate on the level of the flit (flow control unit)
  - Each flit consists of a 72-bit message payload and an 8-bit error control code called a *cyclic redundancy check* (CRC)
- Flow control function
  - Needed to ensure that a sending QPI entity does not overwhelm a receiving QPI entity by sending data faster than the receiver can process the data and clear buffers for more incoming data

Error control function

- Detects and recovers from bit errors, and so isolates higher layers from experiencing bit errors

© 2018 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

QPI Routing and Protocol Layers

Routing Layer

- Used to determine the course that a packet will traverse across the available system interconnects
- Defined by firmware and describe the possible paths that a packet can follow

Protocol Layer

- Packet is defined as the unit of transfer
- One key function performed at this level is a cache coherency protocol which deals with making sure that main memory values held in multiple caches are consistent
- A typical data packet payload is a block of data being sent to or from a cache

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Peripheral Component Interconnect (PCI)

- A popular high bandwidth, processor independent bus that can function as a mezzanine or peripheral bus
- Delivers better system performance for high speed I/O subsystems
- PCI Special Interest Group (SIG)
  - Created to develop further and maintain the compatibility of the PCI specifications
- PCI Express (PCIe)
  - Point-to-point interconnect scheme intended to replace bus-based schemes such as PCI
  - Key requirement is high capacity to support the needs of higher data rate I/O devices, such as Gigabit Ethernet
  - Another requirement deals with the need to support time dependent data streams

Pearson
Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Figure 3.21

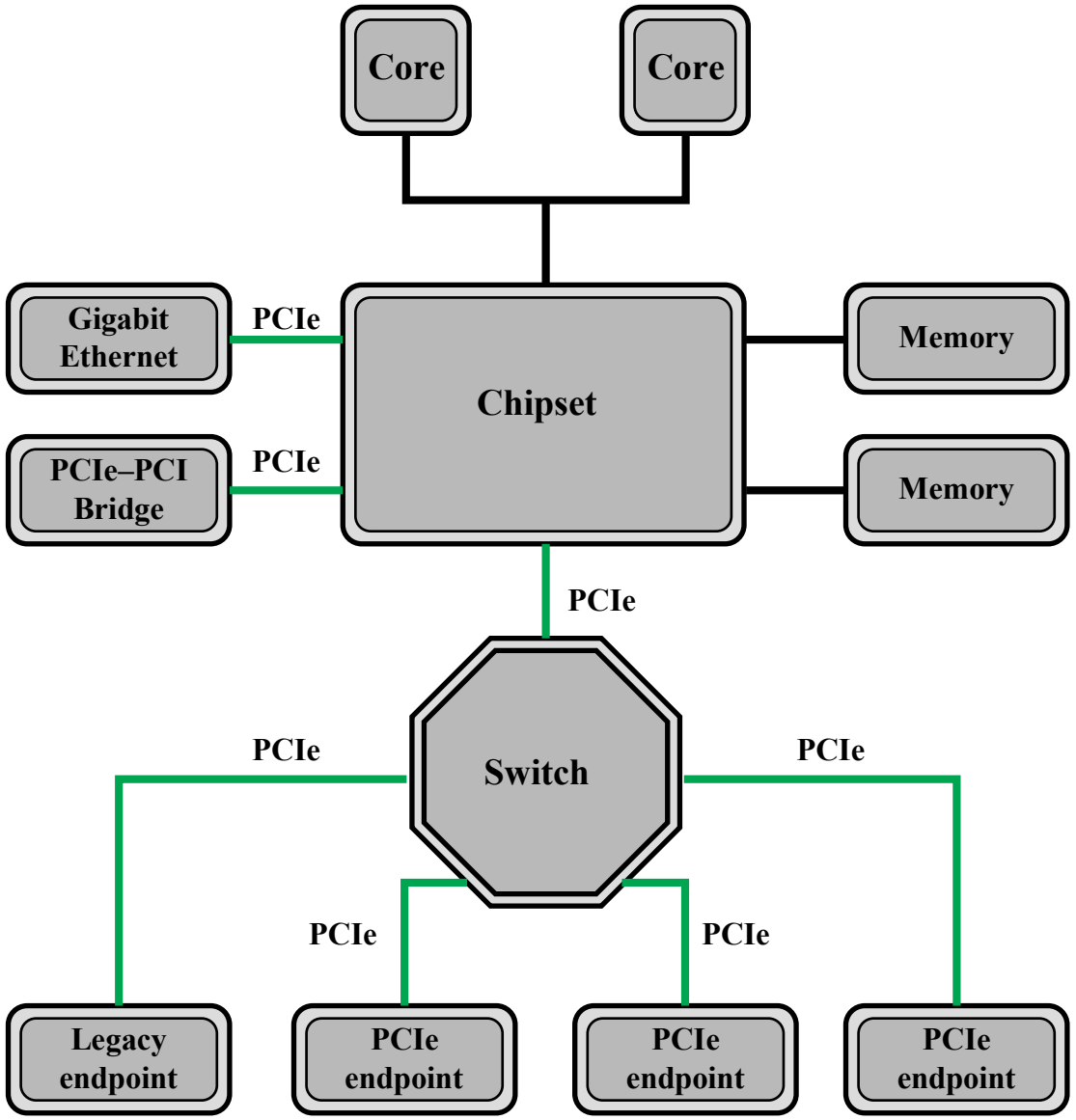


Figure 3.21 Typical Configuration Using PCIe

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Figure 3.22

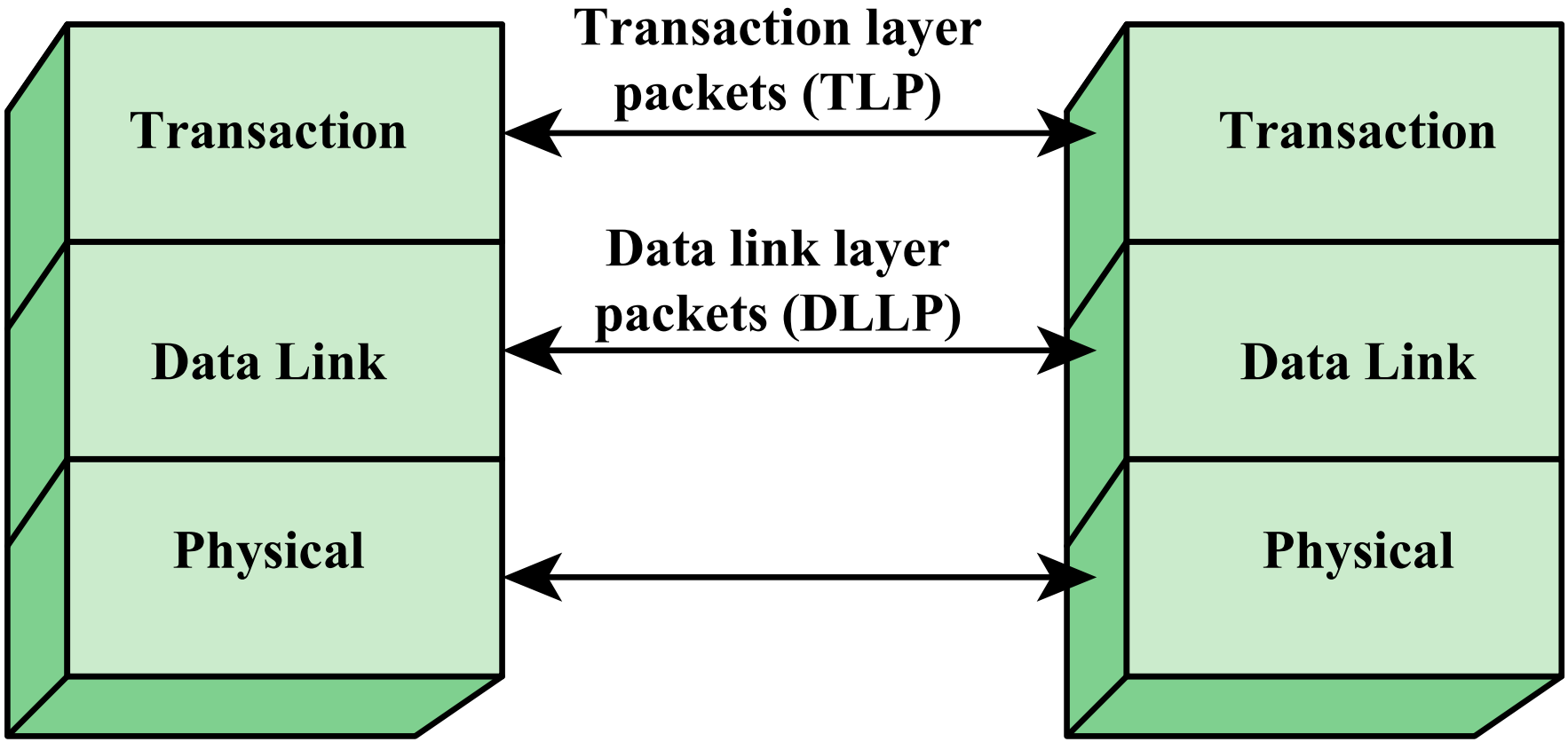


Figure 3.22 PCIe Protocol Layers

Pearson
Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Figure 3.23

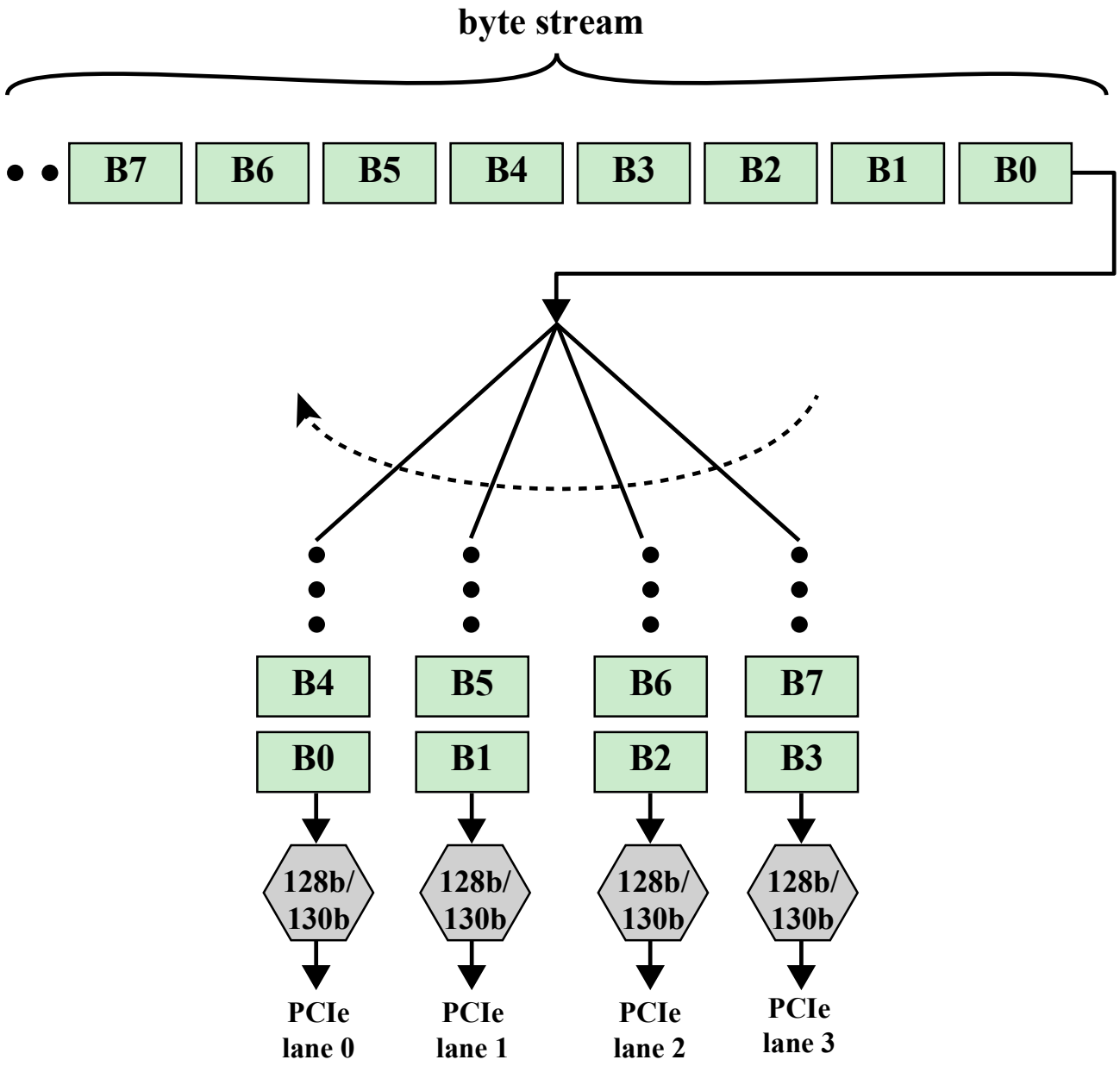


Figure 3.23 PCIe Multilane Distribution

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Figure 3.24

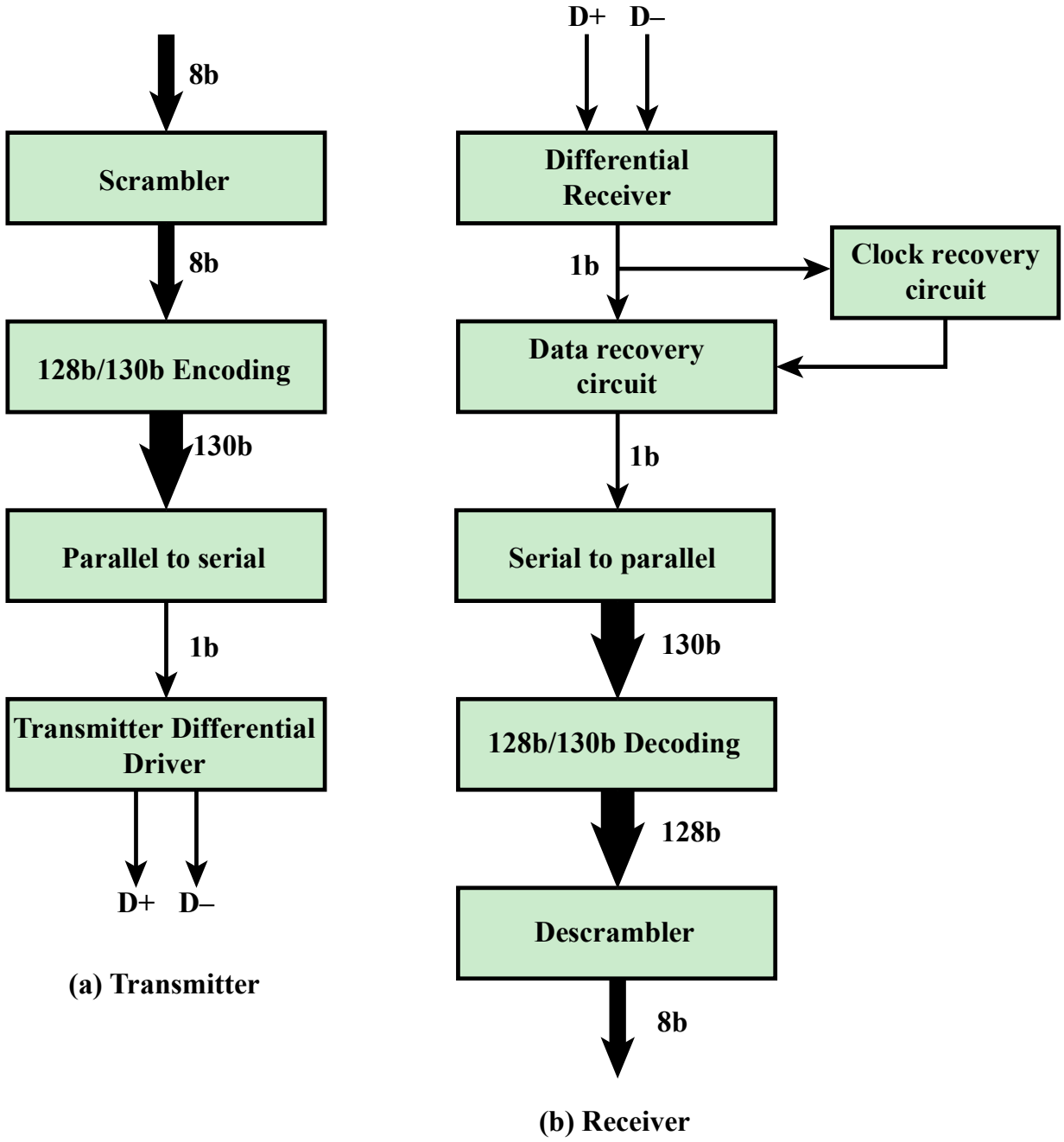


Figure 3.24 PCIe Transmit and Receive Block Diagrams

Pearson
Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

PCIe Transaction Layer (TL)

- Receives read and write requests from the software above the TL and creates request packets for transmission to a destination via the link layer
- Most transactions use a *split transaction* technique
  - A request packet is sent out by a source PCIe device which then waits for a response called a *completion* packet
- TL messages and some write transactions are posted transactions (meaning that no response is expected)
- TL packet format supports 32-bit memory addressing and extended 64-bit memory addressing

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

The TL supports four address spaces:

- Memory
  - The memory space includes system main memory and PCIe I/O devices
  - Certain ranges of memory addresses map into I/O devices
- Configuration
  - This address space enables the TL to read/write configuration registers associated with I/O devices
- I/O
  - This address space is used for legacy PCI devices, with reserved address ranges used to address legacy I/O devices
- Message
  - This address space is for control signals related to interrupts, error handling, and power management

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Table 3.2
PCIe TLP Transaction Types

Address Space	TLP Type	Purpose
Memory	Memory Read Request	Transfer data to or from a location in the system memory map.
	Memory Read Lock Request	
	Memory Write Request	
I/O	I/O Read Request	Transfer data to or from a location in the system I/O Write Request memory map for legacy devices.
	I/O Write Request	
Configuration	Config Type 0 Read Request	Transfer data to or from a location in the configuration space of a PCIe device.
	Config Type 0 Write Request	
	Config Type 1 Read Request	
	Config Type 1 Write Request	
Message	Message Request	Provides in-band messaging and event reporting.
	Message Request with Data	
Memory, I/O, Configuration	Completion	Returned for certain requests.
	Completion with Data	
	Completion Locked	
	Completion Locked with Data	

Pearson
Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Figure 3.25

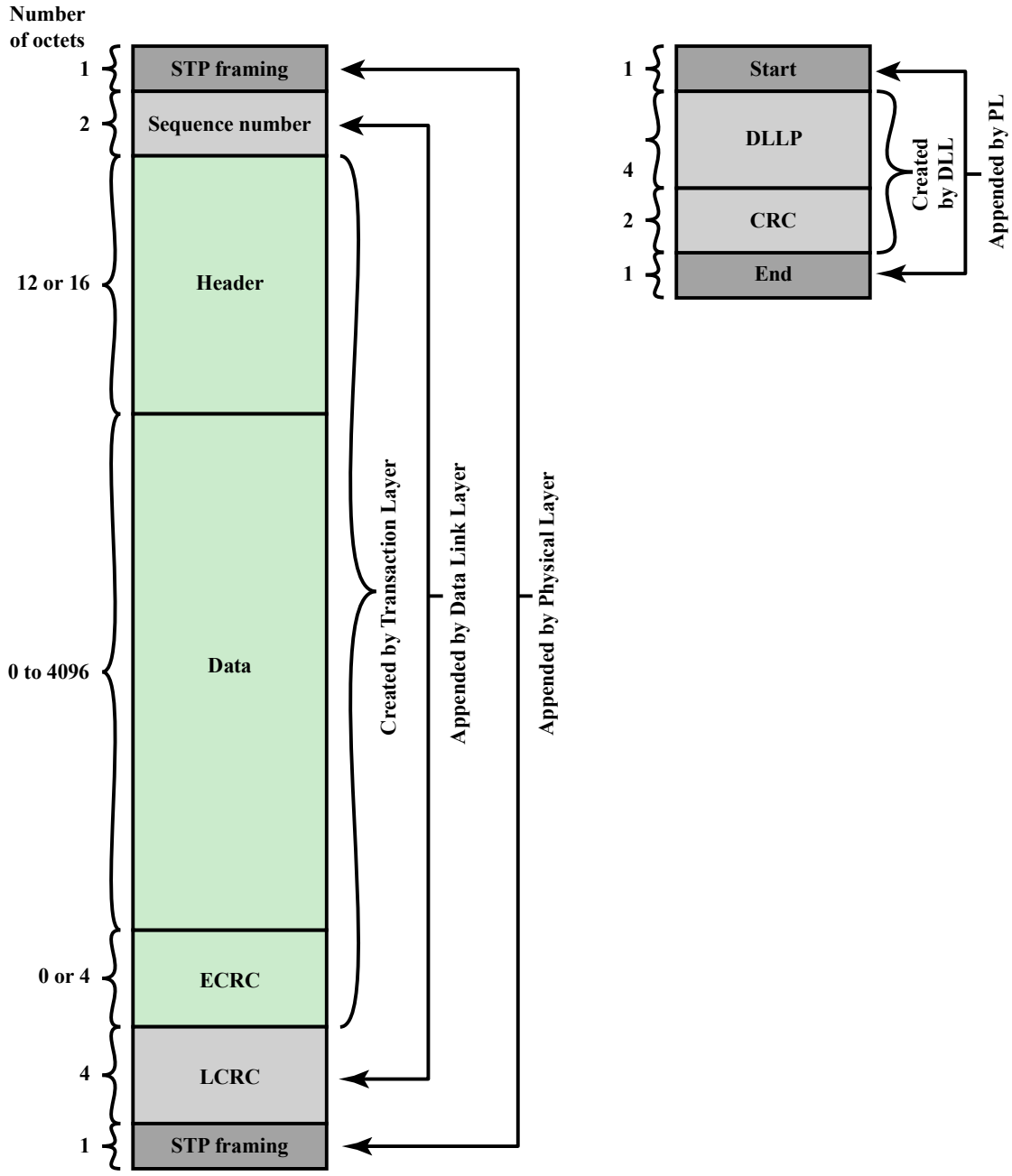


Figure 3.25 PCIe Protocol Data Unit Format

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Summary

Chapter 3

- Computer components
- Computer function
  - Instruction fetch and execute
  - Interrupts
  - I/O function
- Interconnection structures
- Bus interconnection

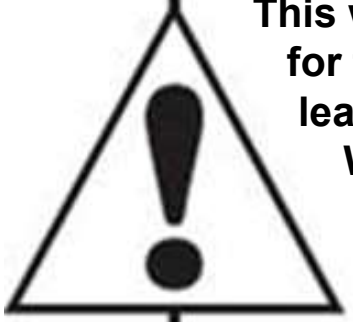
A Top-Level View of Computer Function and Interconnection

- Point-to-point interconnect
  - QPI physical layer
  - QPI link layer
  - QPI routing layer
  - QPI protocol layer
- PCI express
  - PCI physical and logical architecture
  - PCIe physical layer
  - PCIe transaction layer
  - PCIe data link layer

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved

Copyright



This work is protected by United States copyright laws and is provided solely for the use of instructions in teaching their courses and assessing student learning. dissemination or sale of any part of this work (including on the World Wide Web) will destroy the integrity of the work and is not permitted. The work and materials from it should never be made available to students except by instructors using the accompanying text in their classes. All recipients of this work are expected to abide by these restrictions and to honor the intended pedagogical purposes and the needs of other instructors who rely on these materials.

Pearson

Copyright © 2019, 2016, 2013 Pearson Education, Inc. All Rights Reserved